

yugabyteDB query plan management (qpm)

sales.person@yugabyte.com

dan.farrell@yugabyte.com

version 0.03



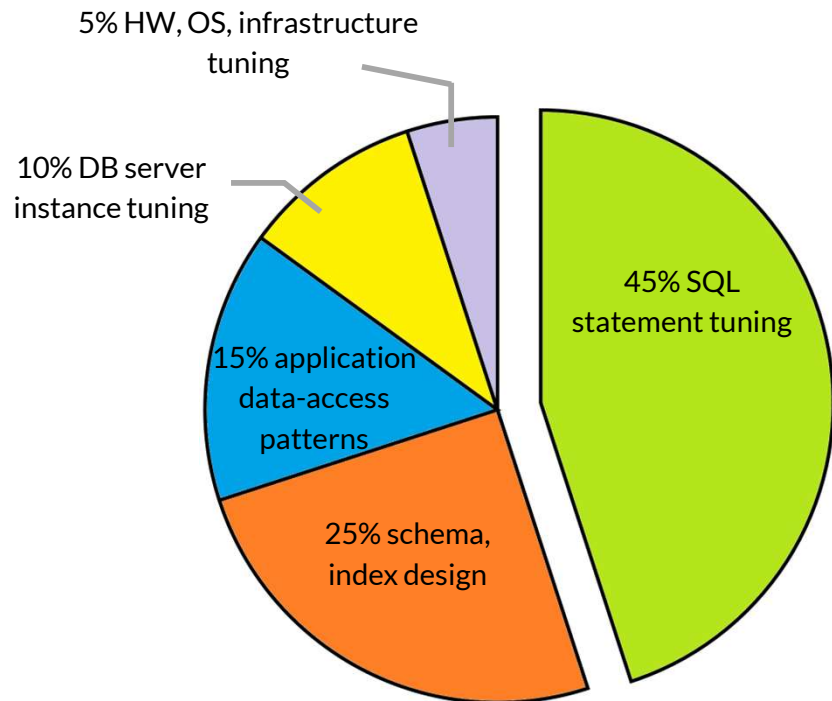
Objectives for today; query plan mgmt

- why:
- the dark arts, secret sauce, voodoo magic of relational databases
 - regularly move queries from 5 minutes to sub-second
 - big (and distributed) systems .. issues get hidden

- what:
- where this fits in performance tuning
 - **a full query optimizer primer; no pre-requisites, [any database]**
 - the 4 stage backend
 - rules based optimizers, cost based
 - examples
 - negations, rewrites
 - should know basic SQL SELECT
 - **finally, query plan management**

- demonstration:
- alerting on regressions

where this fits in performance tuning



bad predicates, missing WHERE-clause **sargability**, unnecessary subqueries, wrong join strategy. consistently the single biggest lever. a single bad query can dominate total load regardless of how well everything else is tuned

normalization level, missing/redundant indexes, partitioning strategy, data types sized appropriately. you can't fix a full table scan with a better-written query if the index simply doesn't exist

$$45\% + 25\% = 70\%$$

N+1 query patterns from ORMs, missing connection pooling, chatty round-trips, lack of batching/caching, wrong isolation level for the workload

memory allocation, buffer/cache sizing, background process config, checkpoint/vacuum tuning, cost-based optimizer settings

storage IOPs, network latency, CPU/RAM sizing, OS-level tuning

sargability – Search ARGument availABLE

Examples;

```
.. WHERE YEAR(order_date) = 2026
.. WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01'

.. WHERE price * 1.08 > 100
.. WHERE price > 100 / 1.08

.. WHERE cust_id = '1234'      -- implicit type cast conversion
.. WHERE cust_id = 1234
```



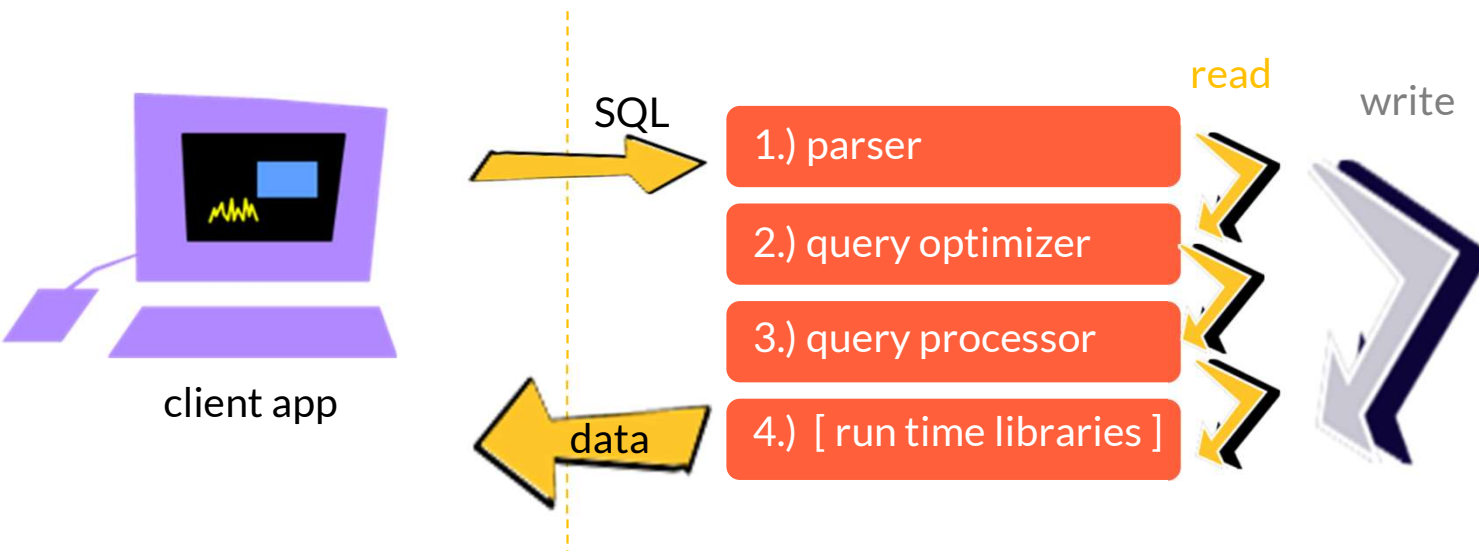
index negation: an existing, perfectly good index can not, or should not be used

which of these-

	reads ?	writes ?
SELECT		
INSERT		
UPDATE		
DELETE		

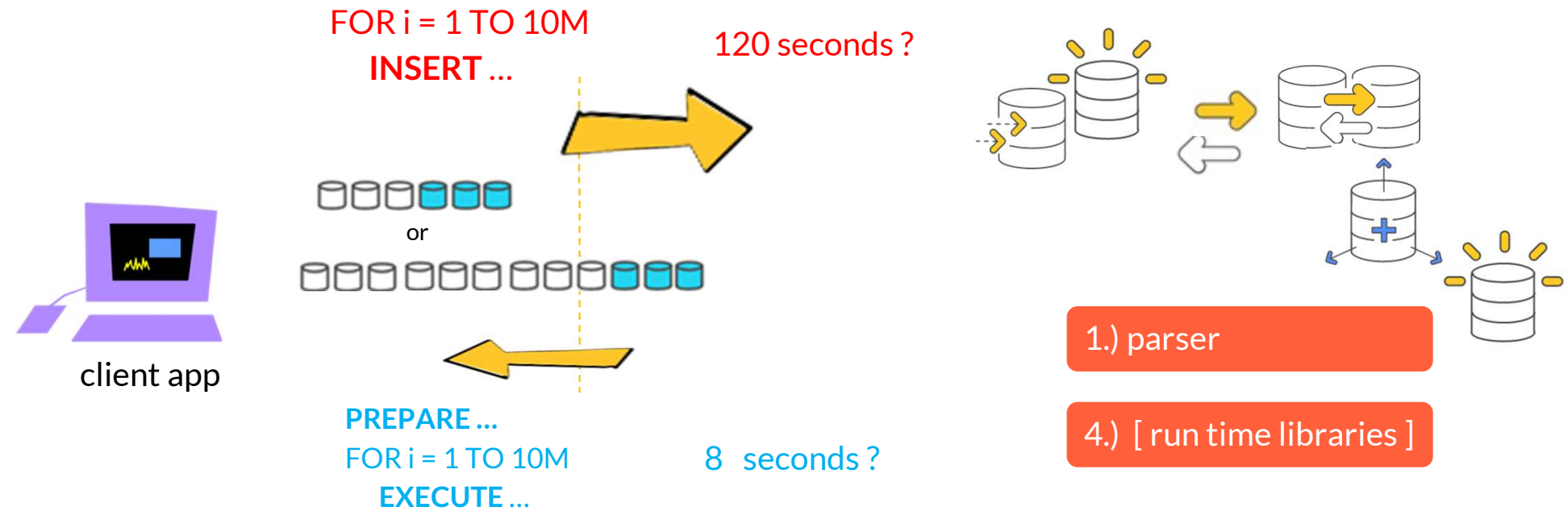
reads: processing, identifying data that is already present

the 4 stage backend



- SQL, declarative
- CREATE TABLE (etc) ..
- or,
 - SELECT .. WHERE
 - UPDATE .. WHERE
 - DELETE .. WHERE
- PREPARE
- or the driver, automatic preparation
- Parsing phase ?
 - can be 85% catalog work
 - authenticate, authorize

seeing parsing overhead



query optimizer; what constitutes a query (execution) plan

- **file (table) access method**
- **join method**
- ***table order***
- predicate pushdown, filter placement
- aggregation strategy
- sort elimination (sort pipelining)
- subquery transformation
- materialization versus pipe lining
- parallel execution planning
- partition pruning
- constants folding, expression simplification
- view/CTE inlining

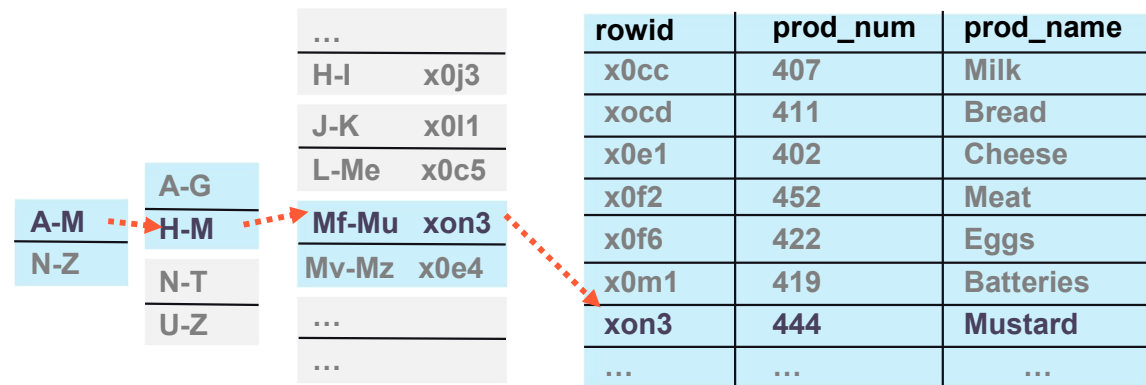
file access methods, (traditionally, all databases)

.. WHERE prod_name = 'Mustard'

index lookup ($O(\log n)$) *much better at scale*

sequential scan ($O(n)$)

rowid	prod_num	prod_name
x0cc	407	Milk
xocd	411	Bread
x0e1	402	Cheese
x0f2	452	Meat
x0f6	422	Eggs
x0m1	419	Batteries
xon3	444	Mustard
...	...	...



index types:

- b-tree+, (implied above) the oldest sql index type, volatile data, equality and range
- hash, second oldest, most performant, equality yes, range no
- lsm, log structured merge-tree, modern choice (index ordered tables)
- gin
- bitmap
- (other)

join method, *table order*

join methods:

- nested loop join (implied here), the oldest
- batch nested loop join (optimized for distributed)
- hash join
- merge join (sort-merge)

cust_num	col_x	order_num
101	aaa	401
102	bbb	402
103	ccc	403
104	ddd	404
105	eee	405
106	fff	406
107	ggg	407
...	...	...



order_num	order_date
401	09/01/16
402	11/12/15
403	07/07/18
404	07/07/18
405	04/04/15
406	03/12/12
407	08/02/18
...	...

rules based optimizer (rbo), cost based optimizer (cbo)

rules based optimizer

- original
- based on sound principles
- **a decision tree**
- 5 rules applied in order to determine query plan
 - outer joins
 - joins
 - filters
 - table size
 - table cardinality



cost based optimizer

- same principles (they were not broken)
- **rbo truths, centered on cost and statistics about the data**

example 01



difficulty

only because it has 3 simple things going on
one join, two predicates, one order by

example 01:

```
select *  
from t1, t2  
where  
    t1.col1 = t2.col1  
and  
    t1.col1 = 5  
and  
    t2.col2 > 5  
order by  
    t2.col2;
```

5 rules applied in order to
determine query plan

- outer joins
- joins
- filters
- table size
- table cardinality

query plan

- file (table) access method
- join method
- *table order*

but first, 7'th grade algebra; the associative property

- applies to addition, multiplication
- not subtraction, division
- in this context,
 - $A = B, B = C, \text{ thus } A = C$

```
select *  
from t1, t2  
where  
    t1.col1 = t2.col1  
and  
    t1.col1 = 5
```

```
and  
    t2.col2 > 5  
order by  
    t2.col2;
```



```
select *  
from t1, t2  
where  
    t1.col1 = t2.col1  
and  
    t1.col1 = 5  
and  
    t2.col1 = 5  
and  
    t2.col2 > 5  
order by  
    t2.col2;
```

also first; the "psuedo order by clause" (a query rewrite)

```
select *  
from t1, t2  
where  
    t1.col1 = t2.col1  
and  
    t1.col1 = 5  
and  
    t2.col1 = 5  
and  
    t2.col2 > 5  
order by  
    t2.col1, t2.col2;
```

why-

- one index (from one table)
- pre-sorted data ("pipelined sort")

example 01

solution

example 01: determining the query plan, applying the rules

```
select *  
from t1, t2  
where  
    t1.col1 = t2.col1  
and  
    t1.col1 = 5  
and  
    t2.col1 = 5  
and  
    t2.col2 > 5  
order by  
    t2.col1, t2.col2;
```

- | | |
|---------------------|--|
| • outer joins | not present |
| • joins | all tables joined (common, 2 nd table+) |
| • filters | 98% of the time (determines first table) |
| • table size | (not needed) |
| • table cardinality | (not needed) |

filter → join → sort FJS

worse table first

make the nested, repetitive action the most efficient

huh?

the nested table will likely have the best access method

as efficiently as possible, reduce the number of tuples to be processed

example 01: which query predicate (filter)

```
select *  
from t1, t2  
where  
    t1.col1 = t2.col1  
and  
    t1.col1 = 5  
and  
    t2.col2 > 5  
order by  
    t2.col2;
```

5 rules applied in order to
determine query plan

- outer joins
- joins
- filters
- table size
- table cardinality

(estimating) filter selectivity

- equality on unique index, 1 tuple
- equality on duplicate index, heuristic, 100 tuples
- range on duplicate index, heuristic/complicated, historically it was 1/3 of tuples
- (we) run queries to count and measure

example 01: how, why

```
...  
where  
  t1.col1 = t2.col1  
and  
  t1.col1 = 5  
and  
  t2.col1 = 5  
and  
  t2.col2 > 5  
order by  
  t2.col1, t2.col2;
```

- best access method: indexed read into t2 (col1, col2)
- (table order set, t2 → t1)
- best access method: indexed read into t1 (col1)
- SQL, table relationships (cardinality)
 - 1:1, M:M, 1:M, 98% are 1:M
 - t1 was our parent, t2 a child (count the join pairs)
- join method
 - determined (supported by) available indexes
 - small tables can alter this

example 01: the actual code behind this

```
-- Explicit HASH primary key (yugabyteDB's preferred PK syntax) rather
-- than the bare "col1 INT PRIMARY KEY" shorthand, which defaults to a
-- range-sharded (ASC) index here. col1 is a monotonically increasing
-- sequence (1..100,000) -- range sharding on that would hotspot writes
-- onto one tablet range at a time; hashing spreads rows evenly across
-- tablets by hash(col1) instead.
```

```
CREATE TABLE t1 (
    col1  INT,
    col2  INT,
    col3  TEXT,
    PRIMARY KEY (col1 HASH)
);
```

```
-- 100,000 rows, col1 = 1..100,000 (unique -- the join key)
```

```
INSERT INTO t1 (col1, col2, col3)
SELECT g, 1 + (g % 1000), repeat('x', 50)
FROM generate_series(1, 100000) AS g;
```

example 01: the actual code behind this

```
-- Composite primary key (col1, col2): col1 carries the equality
-- join/filter (t1.col1 = t2.col1 = 5), col2 the range filter (> 5) --
-- putting both in the PK lets the planner satisfy the WHERE clause with
-- a single index scan instead of a 1,000,000-row seq scan, AND return
-- rows already in col2 order within that col1 value, which can let it
-- skip the separate Sort step for "order by t2.col2" too.
--
-- col1 leads with HASH for the same reason as t1's PK above -- its
-- generator cycles 1..100,000 repeatedly, so a range-sharded leading
-- column would hit tablets in the same moving-sequential-hotspot
-- pattern. col2 stays ASC (range): the col2-sorted-within-a-col1-value
-- guarantee this key exists for lives inside each hash bucket, so
-- hashing col1 doesn't disturb it.
--
-- (col1, col2) is guaranteed unique by construction below -- the 10
-- rows sharing a given col1 land 100,000 generate_series steps apart,
-- and 100,000 mod 997 (col2's modulus, chosen prime) is nonzero, so
-- those 10 rows' col2 values never collide.
CREATE TABLE t2 (
    col1 INT,
    col2 INT,
    col3 TEXT,
    PRIMARY KEY (col1 HASH, col2 ASC)
);
...
```

example 01: the actual code behind this

```
...

-- 1,000,000 rows, col1 cycles 1..100,000 so every t1.col1 value has
-- exactly 10 matching rows in t2 -- guarantees the exercise query's join
-- actually returns rows (t1.col1 = 5 alone matches 10 rows in t2).
--
-- col2 uses modulus 997 (prime), NOT 1000: 1000 divides evenly into
-- 100,000, so a col1-cycle-aligned modulus for col2 would make col2
-- fully determined by col1 -- every row sharing a col1 value would get
-- the identical col2 value, and "t2.col2 > 5" would then either match
-- all or none of them instead of a realistic subset.
INSERT INTO t2 (col1, col2, col3)
SELECT 1 + (g % 100000), 1 + (g % 997), repeat('x', 50)
FROM generate_series(1, 1000000) AS g;
```

example 01: running the original query

```
EXPLAIN (ANALYZE)
select *
from t1, t2
where
    t1.col1 = t2.col1
and
    t1.col1 = 5
and
    t2.col2 > 5
order by
    t2.col2;
```

example 01: output from ANALYZE

QUERY PLAN

```
Nested Loop  (cost=42.19..43.56 rows=10 width=118) (actual time=1.554..1.567 rows=9 loops=1)
->  Index Scan using t2_pkey on t2  (cost=21.10..22.19 rows=10 width=59) (actual time=0.927..0.929 rows=9 loops=1)
      Index Cond: ((col1 = 5) AND (col2 > 5))
->  Materialize  (cost=21.09..21.25 rows=1 width=59) (actual time=0.069..0.069 rows=1 loops=9)
      ->  Index Scan using t1_pkey on t1  (cost=21.09..21.25 rows=1 width=59) (actual time=0.613..0.615 rows=1 loops=1)
            Index Cond: (col1 = 5)
```

Planning Time: 12.405 ms

Execution Time: 2.035 ms

Peak Memory Usage: 202 kB

(9 rows)

- associative principle automatically applied
- pseudo order-by clause automatically applied, pipelined sort
 - *how do we know that* (next page)
- why not
 - batch nested loop join (small data volumes)
 - hash join (we had indexes on both sides)
 - sort-merge (we had indexes, and high selectivity)

example 01b: a different version, that needs to sort

QUERY PLAN

```
Sort (cost=47.03..47.11 rows=30 width=118) (actual time=2.454..2.456 rows=29 loops=1)
  Sort Key: t2.col2
  Sort Method: quicksort Memory: 29kB
-> YB Batched Nested Loop Join (cost=42.19..46.30 rows=30 width=118) (actual time=2.081..2.100 rows=29 loops=1)
  Join Filter: (t1.col1 = t2.col1)
  -> Index Scan using t1_pkey on t1 (cost=21.09..21.74 rows=3 width=59) (actual time=1.305..1.307 rows=3 loops=1)
      Index Cond: (col1 = ANY ('{5,6,7}'::integer[]))
  -> Index Scan using t2_pkey on t2 (cost=21.10..24.56 rows=30 width=59) (actual time=0.747..0.754 rows=29 loops=1)
      Index Cond: ((col1 = ANY (ARRAY[t1.col1, $1, $2, ..., $1023])) AND (col2 > 5))
Planning Time: 15.893 ms
Execution Time: 3.049 ms
Peak Memory Usage: 641 kB
(12 rows)
```

yugabyteDB doc page: Ask AI capability

docs.yugabyte.com

 Yugabyte Docs AI

Ask

Search

Use MCP ↕

This is a custom LLM for Yugabyte with access to developer docs (docs.yugabyte.com), and all GitHub issues from the past year (github.com/yugabyte/yugabyte-db/issues). Please note that answers are generated by AI and may not be fully accurate.

which index best serves this query,
select *
from t1, t2
where
 t1.col1 = t2.col1
and
 t1.col1 = 5
and
 t2.col2 > 5
order by
 t2.col2;

Index Design for Your Join Query

Your query is:

```
SELECT *  
FROM t1, t2  
WHERE t1.col1 = t2.col1  
AND t1.col1 = 5  
AND t2.col2 > 5  
ORDER BY t2.col2;
```

Let's break down the access pattern:

- `t1.col1 = 5` — equality predicate on `t1`
- `t2.col1 = t2.col1` joined with `t1.col1 = 5`, so effectively `t2.col1 = 5` — equality predicate on `t2`
- `t2.col2 > 5` — range predicate on `t2`
- `ORDER BY t2.col2` — sort on `t2.col2`

Recommended Indexes

Following the **ESR (Equality → Sort → Range)** rule and the general column ordering principles for YugabyteDB:
🔗 ESR rule 🔗 column ordering

Index on `t1`

Since `t1` is only accessed via an equality predicate (`col1 = 5`), a simple index suffices:

```
CREATE INDEX ON t1 (col1 ASC);
```

If you need a covering index to avoid table fetches, include any other columns selected from `t1`:

```
CREATE INDEX ON t1 (col1 ASC) INCLUDE (/* other selected columns */);
```

more on sort pipelining

```
select *  
from t1  
where  
    col1 = 'X'  
and  
    col2 = 'Y'  
order by  
    col3;
```

- which single index is best ?
 - what is the column order ?
- given an index on (col1, col3) that returns 100,000 rows and an index on (col2) that returns 100 rows which access method is better ?
- what if the application (pages) and the user rarely reads more than 5 rows ?



checkpoint

- ballpark; where can we make a 5 minute query run in sub-second ? (network tuning, OS tuning, ..)
- what is index negation ? give an example
- besides SELECT, name two query verbs that read
- name the 4 stages in the backend that queries go thru
- which two does CREATE TABLE experience ? why ?
- how can you bypass the parser ? why would you want to ?
- what are the 3 primary elements of a query plan and which one pretty much drives the whole process ?
- name two file access methods
- name two join methods
- our two table query, both tables with predicates; what additional two query topics did we introduce here ? briefly describe each
- .. WHERE col1 = 'FFF' and col2 > 20;
knowing nothing else, which predicate returns more rows ? which predicate do we prefer to process first ?
- what is sort pipelining ?

example 01

hands on exercise

- exercise 01
- exercise 01b (shows sort)

example 02



difficulty

one new index negation, one new query rewrite

one table, two uncompounded predicates

example 02:

```
select *  
from t1  
where  
    col1 = 10  
or  
    col2 = 20;
```

5 rules applied in order to
determine query plan

- outer joins
- joins
- filters
- table size
- table cardinality

query plan

- file (table) access method
- join method
- *table order*

the OR-topped query, the physics

```
select *  
from population  
where  
    age = 30  
or  
    state = 'CO';
```

tuples could include-

- age = 30 and state = 'WI'
- age = 10 and state = 'CO'
- neither
- both age = 30 and state = 'CO'

how your query process can not affect the result set

huh?

a given tuple can not be returned twice (in error) because of your chosen query processing

index negation; non-anchored composite key

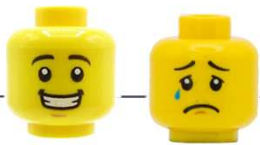
	lname	fname
	Smite	Carly
	Smith	Alan
	Smith	Carly
	Smith	Dave
	Smith	Jim
	Smithson	Naitik
	Stang	Billie
	...	...

phone book

- sorted (composite indexed) descending by last name, then first name descending
- find all persons with first name, Carly
- first name is not the first column, not the anchor column
 - index is now unanchored (data is not sorted by first name)
 - more performant to be a sequential scan
- unless it is key-only (discussed later, but doesn't help us here)

there is no single index serving (anchored) by both columns

composite key, index negation chart



col1		col2		col3		
=		=		=		
=		=				
=						
=		>		=		col2, if defined as ASC, col3 not used for query
<> NOT						based on expected selectivity
=		<> NOT				col2 based on selected selectivity
		=	 			if the retrieval is secondary key only, but ..
						...

OR-topped query → OR-union transfer (a query rewrite)

```
select *  
from t1  
where  
    col1 = 10  
or  
    col2 = 20;
```



```
select * from t1 where  
where col1 = 10  
union all  
select * from t1  
where col2 = 20;
```



```
select * from t1 where  
where col1 = 10  
union  
select * from t1  
where col2 = 20;
```

when-

- both predicates are indexed
- both predicates are selective

use union all when

- both predicates produce a unique result set
- most performant

use union when

- there might be duplicates in the two result sets
- (sorts both lists, merge, remove duplicates), blecht

example 02

solution

example 02: determining the query plan, applying the rules

```
select *  
from t1  
where  
    col1 = 10  
or  
    col2 = 20;
```

- outer joins not present
- joins all tables joined (common, 2nd table+)
- filters 98% of the time (determines first table)
- table size (not needed)
- table cardinality (not needed)

filter → join → sort FJS

- only one table; no join methods, no table order, pick a predicate-
- because of the chance a row would qualify for both access methods, we need to sort, merge, remove duplicates
- without a [manual or automatic OR-union transfer](#), historically this would only ever be sequential access method

example 02: the actual code behind this

```
-- Explicit HASH primary key -- see "10 - create exercise 01.sql" for
-- why (col1 is a monotonically increasing sequence; hashing avoids a
-- moving-sequential-hotspot on writes).
CREATE TABLE t1 (
  col1  INT,
  col2  INT,
  col3  TEXT,
  PRIMARY KEY (col1 HASH)
);

-- 100,000 rows, col1 = 1..100,000 (unique), col2 = 1..1,000 (repeating)
-- -- ~100 rows match col2 = 20, plus the single col1 = 10 row.
INSERT INTO t1 (col1, col2, col3)
SELECT g, 1 + (g % 1000), repeat('x', 50)
FROM generate_series(1, 100000) AS g;

-- col2 has the same repeating-cycle shape as col1, so HASH here too --
-- same write-hotspot reasoning as everywhere else in this lab.
CREATE INDEX t1_col2_idx ON t1 (col2 HASH);
```

example 02: running the query

```
EXPLAIN (ANALYZE)
select *
from t1
where
    col1 = 10
or
    col2 = 20;
```

example 02: output from ANALYZE

QUERY PLAN

```
-----  
YB Bitmap Table Scan on t1  (cost=78.55..98.96 rows=101 width=59) (actual time=2.594..2.605 rows=101 loops=1)  
  
->  BitmapOr  (cost=54.46..54.46 rows=101 width=0) (actual time=0.954..0.954 rows=101 loops=1)  
    ->  Bitmap Index Scan on t1_pkey  (cost=0.00..21.22 rows=1 width=0) (actual time=0.612..0.612 rows=1 loops=1)  
        Index Cond: (col1 = 10)  
    ->  Bitmap Index Scan on t1_col2_idx  (cost=0.00..33.19 rows=100 width=0) (actual time=0.338..0.338 rows=100 loops=1)  
        Index Cond: (col2 = 20)
```

Planning Time: 11.312 ms
Execution Time: 3.253 ms
Peak Memory Usage: 190 kB
(9 rows)

- because both predicates are supported via indexes, the OR-union transfer functional equivalent is automatically applied (a query rewrite)
- other access methods output tuples. bitmap index scan outputs tuple ids (not column values proper). this means we will need to go also to the data page once the result sets are formed. (not a candidate for covering indexes, discussed later.)
- bitmap index scan supports efficient; BitMapOr(s), BitMapAnd(s)
- BitMapOr, grossly efficient means to intersect two result sets
- *more notes on next page-*

example 02: additional topics

to be aware of-

- this is named OR "topped", but it applies to any OR inside a query at any level in the WHERE clause. all ORs cause this branching.
- `!=` and `<>` can, in given cases, be transformed to an `OR`
- currently, if this doesn't fit in `work_mem`, this automatic query rewrite will fall back to sequential scan
- `work_mem`
 - all databases have this; (working) memory or virtual memory (temporary tables, files)
 - default 4MB, one allocation per each of sort/hash/aggregate/node/query
 - `SET work_mem = '64MB';` -- session only, 'session level setting'
 - `SET LOCAL work_mem = '64MB';` -- just this one transaction
 - `ALTER DATABASE my_db SET work_mem = '64MB';` -- per database
 - `ALTER ROLE my_role SET work_mem = '64MB';` -- per user
 - `ysql_pg_conf_csv, work_mem=64MB` -- yb-tserver

example 02'ish: a different query, seeing work_mem and sorts

```
SET work_mem = '16MB';
EXPLAIN (ANALYZE)
select *
from t1
order by
    col2;
```

QUERY PLAN

```
-----
Sort  (cost=21562.22..21812.22 rows=100000 width=59) (actual time=267.715..279.740 rows=100000 loops=1)
  Sort Key: col2
  Sort Method: quicksort  Memory: 13229kB
```

```
-> Seq Scan on t1  (cost=21.09..13257.40 rows=100000 width=59) (actual time=2.579..241.263 rows=100000 loops=1)
```

```
Planning Time: 10.982 ms
Execution Time: 293.255 ms
Peak Memory Usage: 19755 kB
(7 rows)
```

example 02'ish: a different query, seeing work_mem and sorts

```
SET work_mem = '4MB';
```

```
EXPLAIN (ANALYZE)
```

```
select *  
from t1  
order by  
    col2;
```

```
QUERY PLAN
```

```
-----  
Sort  (cost=25324.72..25574.72 rows=100000 width=59) (actual time=266.545..271.658 rows=100000 loops=1)  
  Sort Key: col2  
  Sort Method: external merge  Disk: 6768kB
```

```
-> Seq Scan on t1  (cost=21.09..13257.40 rows=100000 width=59) (actual time=2.290..234.344 rows=100000 loops=1)
```

```
Planning Time: 10.003 ms
```

```
Execution Time: 285.005 ms
```

```
Peak Memory Usage: 5238 kB
```

```
(7 rows)
```

```
// what ??
```

example 02'ish: a different query, sort time jitter on small sized data

The Sort node's own actual time=X..Y numbers mix two different things: X (time to first row) already includes the child Seq Scan's full time plus the sort itself, since Sort is a blocking operator.

So Execution Time differences between runs were mostly driven by the Seq Scan underneath — which is identical logic either way and, on this shared cluster, varied noisily between 237ms and 289ms across otherwise-identical runs, dwarfing whatever the sort itself costs.

Subtracting the scan's own time from the sort's start time isolates the actual sort cost:

run	in-memory quicksort	external merge (disk)
1	29.7ms	32.5ms
2	25.9ms	34.0ms
3	26.7ms	36.4ms
avg	~27.4ms	~34.3ms

Once the scan noise is stripped out, the disk-spilling sort is consistently slower, as expected — quicksort has no I/O to pay for.

The one-off result in the slides where the disk version looked faster was Execution Time being dominated by scan-time jitter on the shared cluster, not anything about the sort method itself.

Total Execution Time isn't a reliable way to compare these two — the scan noise (tens of ms) is larger than the real sort-method difference (~7ms).

example 02: optional examination during exercise

as an optional exercise only

- disable bitmap scans
 - SET enable_bitmapscan = off;
 - SET yb_enable_bitmapscan = off;
- run the OR-topped query as is
- do your own OR-union transfer

checkpoint

- which index negation was introduced in this example
- which query rewrite was introduced in this example
 - what two conditions needs to be in place in order to use this rewrite
- what is the difference between UNION and UNION ALL
- where does work_mem go if it overflows
- the default work_mem is 4MB; on one node, how much memory might you use for this query-

```
select
  col2,
  count(*) as row_count
from t1
group by col2
order by row_count desc
```

example 02

hands on exercise

- exercise 02
- exercise 02b (work_mem)
- exercise 02c (work_mem)
- exercise- bitmap scan off, no code

discussion

more detail on indexes, index types

index types

- all enterprise databases have different index types
 - for permanent tables, possibly different index types for temporary tables/files
 - (implied) and possibly different index types used only in the execution of a query
 - different index types support different operators (equality, range, wildcard..), and different performance profiles (RAM, disk, volatile/non-volatile data, ..)
- yugabyteDB 2026.1 has 3 index types for permanent tables-
 - LSM, HNSW, GIN
- B-tree+, hash, syntax supported for compatibility
- for geospatial, we can use geohash and LSM

scalars
text
Vector

PostgreSQL/other compatible; B-tree+, hash create index (rewrites)

```
CREATE TABLE am_demo (col1 int, col2 int);
```

```
CREATE INDEX am_demo_btree_idx ON am_demo USING btree (col1);
```

```
-- NOTICE: substituting access method "lsm" for "btree" in YugabyteDB
```

```
CREATE INDEX am_demo_hash_idx ON am_demo USING hash (col2);
```

```
-- NOTICE: substituting access method "lsm" for "hash" in YugabyteDB
```

```
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'am_demo';
```

```
-- indexname          |          indexdef
```

```
-- -----+-----
```

```
-- am_demo_btree_idx | CREATE INDEX am_demo_btree_idx ON public.am_demo USING lsm (col1 ASC)
```

```
-- am_demo_hash_idx  | CREATE INDEX am_demo_hash_idx ON public.am_demo USING lsm (col2 ASC)
```



index types, focus on LSM

- lsm, log structured merge-tree
 - append-only *write* path, sorted batches, tidies up later, optimized for write
- column by column in create index; hash or asc/desc
 - hash spreads data evenly across nodes, supports equality, table is partitioned/sharded into "tablets"
 - asc/desc does not "spread" data, can allow node hot spots, but does support equality and range
 - real world pattern ?
 - lead index with hash, trail with asc/desc
- there are other partition/sharding techniques, not discussed here-
 - geo-partitioning
 - times series data

examples used previously-

-- example 01

-- t1's primary key

```
CREATE TABLE t1 (  
    col1 INT,  
    col2 INT,  
    col3 TEXT,  
    PRIMARY KEY (col1 HASH)  
);
```

-- t2's primary key (composite)

```
CREATE TABLE t2 (  
    col1 INT,  
    col2 INT,  
    col3 TEXT,  
    PRIMARY KEY (col1 HASH, col2 ASC)  
);
```

-- example 02

-- t1's primary key

```
CREATE TABLE t1 (  
    col1 INT,  
    col2 INT,  
    col3 TEXT,  
    PRIMARY KEY (col1 HASH)  
);
```

what about datetime/timestamp data, the core tension-

sharding	range query on timestamp	write hotspot risk
hash (default)	bad – scatter-gather across all tablets since order is not preserved	low – hash spreads rows evenly
range (pure ASC/DESC on times stamp)	great – only tablets covering the range are touched	bad – monotonically increasing timestamps always land on the last tablet , so all new writes pile onto one node no matter how much tablet splitting happens

a pure range index/PK on (event time) is the number one cause of hot tablets in time-series schemas

every new row has a "greater" key, than everything before it, so it is always routed to the same tail tablet

datetime/timestamp data: the fix (1)

```
CREATE TABLE events (  
  device_id      uuid,  
  event_time     timestampz,  
  payload        jsonb,  
  PRIMARY KEY ((device_id) HASH, event_time DESC)  
);
```

give the key a hash prefix on something high cardinality, and a range suffix on the timestamp

```
WHERE  
  device_id = ?  
AND  
  event_time BETWEEN ? AND ? ;
```

datetime/timestamp data: if you do not have a natural key, the fix (2)

```
ALTER TABLE events ADD COLUMN bucket int  
    GENERATED ALWAYS AS (extract(hour from event_time)::int % 16) STORED;  
  
CREATE INDEX idx_events_bucket_time  
    ON events ((bucket) HASH, event_time DESC) INCLUDE (payload);
```

synthesize a bucket, a time truncated hash

spreads writes across (n) Tablets as hours roll over, instead of one ever-growing tablet

pick a modulus to achieve the desired parallelism

datetime/timestamp data: if you can not do either, the fix (3)

yugabyteDB does a parallel scatter-gather across the hash buckets
the range predicate on (event time) still gets pushed down into each tablet. each tablet scans its own slice
instead of a full scan

trades 1 tablet for a (cheaper pruned range scan in parallel), usually a net win

additionally:

- pre-split the tablets at table creation (SPLIT INTO n TABLETS), to get immediate parallelism, instead of waiting for automatic tablet splitting to catch up
- use covering indexes

similar, but different: permitting NULLs in indexes, the fix

NULL is just another value to a hash function

if 30-40% of the data in the leading composite index column are NULL, these all hash to the same bucket, creating a hot tablet just like any other skewed value

```
CREATE TABLE events (  
  device_id      uuid,  
  optional_tag text,          -- frequently NULL, currently  
                               -- the hotspot offender  
  event_time     timestampz,  
  payload        jsonb  
);  
  
-- only index rows where optional_tag is actually populated  
--  
CREATE INDEX idx_events_tag_time  
  ON events ((optional_tag) HASH, event_time DESC)  
  WHERE optional_tag IS NOT NULL;
```

example 03



difficulty

another index negation, a new index type
one table, one predicate

example 03:

```
select *  
from t1  
where  
  col1 LIKE '%xxx'
```

5 rules applied in order to
determine query plan

- outer joins
- joins
- filters
- table size
- table cardinality

query plan

- file (table) access method
- join method
- *table order*

index negation; non-initial substring

	Iname	colx
	Caldera	aaa
	Flynn	bbb
	Johnson	aaa
	Knicely	ddd
	Paneendra	eee
	Schulte	bbb
	Williamson	ccc
	...	...

non-initial substring (non-initial anything), solutions

- why-
 - country → province → city
city → province → country
especially ISV/COTs applications
 - lead with selectivity
 - or; is this non-initial (data) actually an attribute
- reverse index (meh)
- **GIN indexes**

reverse index, syntax and use

```
CREATE TABLE t2 (  
    id          BIGINT,  
    coll        TEXT,  
    coll_rev    TEXT GENERATED ALWAYS AS (reverse(coll)) STORED,  
    PRIMARY KEY (id HASH)  
);  
  
-- Plain ascending LSM index on the reversed text -- ordinary prefix  
-- range scan support, nothing exotic needed.  
CREATE INDEX t2_coll_rev_idx ON t2 (coll_rev ASC);  
  
ANALYZE t2;
```

reverse index, syntax and use

```
-- original query
SELECT * FROM t1 WHERE col1 LIKE '%GE AK';

-- QUERY PLAN
-- -----
-- Seq Scan on t1 (cost=21.10..61958.72 rows=9851 width=41)
--      (actual time=74.319..303.490 rows=5000 loops=1)
--      Storage Filter: (col1 ~~ '%GE AK'::text)
--      Planning Time: 41.525 ms
--      Execution Time: 317.859 ms
--      Peak Memory Usage: 81 kB
```

reverse index, syntax and use

```
-- query that uses reverse index
SELECT * FROM t2 WHERE coll_rev LIKE 'KA EG%';

-- QUERY PLAN
-- -----
-- Index Scan using t2_coll_rev_idx on t2  (cost=48.20..48.49 rows=9851 width=74)
--      (actual time=19.407..69.373 rows=5000 loops=1)
--    Index Cond: ((coll_rev >= 'KA EG'::text) AND (coll_rev < 'KA EH'::text))
--    Storage Index Filter: (coll_rev ~~ 'KA EG%'::text)
-- Planning Time: 46.123 ms
-- Execution Time: 72.038 ms
-- Peak Memory Usage: 103 kB
```


GIN index, generalized inverted index

- not a TF/IDF (term frequency, inverse document frequency index), ala Solr/Lucene
- *generalized*, decomposes indexed value into set of keys
 - array elements for arrays
 - keys/values for JSONB
 - trigrams for pg_trgm
- *inverted*, meaning; keys, then the list of rows containing it, but it fans in, not out

GIN index, generalized inverted index

```
SELECT show_trgm('Hello World');
--
--          show_trgm
-- -----
--  {"  h","  w"," he"," wo", ell, hel, "ld ", llo, "lo ", orl, rld, wor}

-- UI nit; data is returned sorted, deduplicated, not in source word order
-- pads at blank space boundaries; 2 leading, 1 trailing
```

example 03

solution

example 03: creating a GIN index

```
CREATE TABLE t3 (  
    id      BIGINT,  
    col1    TEXT,  
    PRIMARY KEY (id HASH)  
);  
  
CREATE EXTENSION IF NOT EXISTS pg_trgm;  
  
CREATE INDEX t3_col1_trgm_idx ON t3 USING gin (col1 gin_trgm_ops);  
  
ANALYZE t3;
```

example 03: the query plan

```
EXPLAIN (ANALYZE)
SELECT * FROM t3 WHERE col1 LIKE '%GE AK';

-- QUERY PLAN
-- -----
-- Index Scan using t3_col1_trgm_idx on t3
--      (cost=16.00..25.71 rows=98 width=41)
--      (actual time=23.255..947.165 rows=5000 loops=1)
--    Index Cond: (col1 ~~ '%GE AK'::text)
--    Rows Removed by Index Recheck: 200000
-- Planning Time: 42.658 ms
-- Execution Time: 949.507 ms
-- Peak Memory Usage: 135 kB
```

this test used highly
duplicative/generated data

effectively, the index over-fetches
possible matches, then fully qualifies
rows (rechecks) before returning to
the calling function

a different data set; seeing quality of query predicates, and index rechecks

a trigram index breaks text into 3-character sequences

for example:

"Yummy Donut" creates trigrams: `yum`, `umm`, `mmy`, `my`, `y d`, `do`, `don`, `onu`, `nut`

query 1: ``%Yummy Donut%` - 5,022,410 rechecks (85 sec)

the space in "Yummy Donut" creates the trigram `y d` (y-space-d)

this is an extremely common trigram (any word ending in 'y' followed by word starting with 'd')

index returns 5+ million candidate rows that must be rechecked

query 2: ``%Yumm%Donut%` - 319,070 rechecks (10 sec)

no space between words means different trigrams: `mmd`, `mdo`, `don`

these are ****much less common**** trigrams

far fewer false positives to recheck

...

a different data set; seeing quality of query predicates, and index rechecks

...

query 3: ``%Yumm% Donut%` - 5,022,376 rechecks (86 sec)

has the space, so includes `m d` trigram

again, a very common pattern (words ending in 'm' + space + 'd')

same very large recheck

query 4: ``%umm% %onut%` - 17,114 rechecks (1.3 sec)

lowercase makes it different: `m o` trigram

the double wildcard `% %` with lowercase is less common

much better performance

...

a different data set; seeing quality of query predicates, and index rechecks

...

query 5: ``%Yumm% %Donut%` - 319,070 rechecks (10 sec)

the pattern `% %` (space-space) between wildcards creates different matching
more selective than single space

case-sensitive single-character + space trigrams like `y d`, `m d`

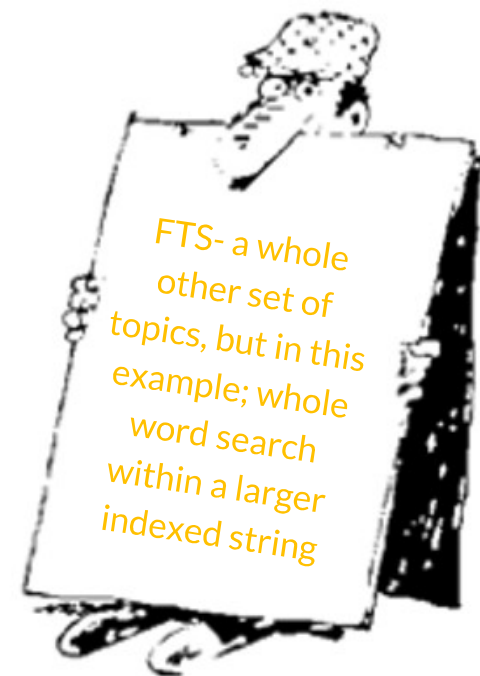
these patterns match millions of unrelated phrases:

"Happy dog", "Very different", "My data", "eby digital", etc

[github issue, #7850](#), a current limitation is that only one scan term is used at a time

what about full text search (FTS), a *different use case*

```
CREATE TABLE t4 (  
    id      BIGINT,  
    coll    TEXT,  
    PRIMARY KEY (id HASH)  
);  
  
CREATE INDEX t4_coll_lex_idx ON t4  
    USING gin (to_tsvector('simple', coll));  
  
ANALYZE t4;  
  
EXPLAIN (ANALYZE)  
SELECT * FROM t4  
WHERE to_tsvector('simple', coll) @@ to_tsquery('simple', 'Golden\ Dragon:*');
```



full text search, query plan

QUERY PLAN

```
-----  
Index Scan using t4_coll_lex_idx on t4  (cost=20.00..35.00 rows=400 width=41)  
  (actual time=9.992..71.995 rows=10000 loops=1)  
    Index Cond: (to_tsvector('simple'::regconfig, coll) @@  
      '''golden':* <-> 'dragon':*':tsquery)  
Planning Time: 9.159 ms  
Execution Time: 73.904 ms  
Peak Memory Usage: 78 kB
```

checkpoint

- for permanent tables, what are the 3 index types available with yugabyteDB 2026.1
- the first index type we detailed has further modifiers/adjectives for columns-
 - what are they named
 - which do you lead with ? why ?
 - which operators do each support
- which index negation was introduced in this example
- describe the workings of a reverse index
- what does GIN index stand for as an acronym
- in addition to trigrams, what two additional (use cases) were listed for GIN indexes
- when using a GIN (trigram) index, describe briefly what an index recheck is
- same data, same column, same GIN index, different search terms; why would one query return in 1 second and the other return in 30 seconds
- briefly describe FTS, full text search

example 03

hands on exercise

- exercise 03
- exercise 03b
- exercise 03c

discussion

random topics (housekeeping, backlog of unresolved dependencies)-

- index count per table-
 - yugabyteDB write path
- (group)
 - collocated indexes
 - index ordered tables
 - key only retrieval, covering indexes

index count, write path

- an index is a vertical slice of a table; generally, all rows and a subset of columns
 - a single row write to a table with 6 indexes can be 7x the write volume
- count of indexes per table for OLTP type systems-
 - olden times, aimed for less than 3 indexes per table (2.6-2.8) on average
 - modern times, it's just additional parallel, asynchronous CPU workload, disk space. not free, but also not the final word towards achieving a given SLA
- modern times
 - we are networked, distributed, horizontally scaled
 - data consistency; strong or eventual
 - synchronous or asynchronous data replication

...

index count, write path

...

given 3 failure domains, 3 nodes each (9 nodes total) , replication factor 3, and a transaction writing 3 rows-

- **placement:** each row hashes to a tablet, each tablet's 3 replicas land one-per-domain — a full domain failure costs that tablet at most 1 replica, so the remaining 2 still form a Raft majority
- **per-row write:** for each of the 3 rows, the tablet leader appends a provisional write (an “intent”) and replicates it to its followers via Raft consensus
- **write pipelining:** without it, each row's replication round would need to be acknowledged before the next row's write is sent. with pipelined writes, the leader accepts and starts replicating the next row's intent before the prior one's round has finished — so the three rows' Raft rounds overlap in flight rather than being serial
- **commit:** once all three intents are durably replicated, the transaction commits by writing a single record to the transaction status tablet, atomically making all three rows visible together
- **net effect:** [three back-to-back network-plus-consensus round trips become three overlapping ones](#) — wall-clock commit time drops roughly in proportion to how much overlap pipelining achieves

index count, write path, with indexes

same 3-row transaction, same 3×3 domain layout, but the table now has 2 secondary indexes-

- **indexes are their own tablets**: each secondary index is itself an independently-sharded table. writing a row means writing not just its base-table tablet, but also one entry per index — and each index entry is hashed by the indexed column's value, so it very likely lands on a different tablet (and different leader/node) than the base row itself
- participant count multiplies: each of the 3 rows now touches up to 3 tablets (1 base + 2 index) instead of 1 — up to 9 participant tablets total for the one transaction, each still independently Raft-replicated across the 3 failure domains
- **write pipelining matters more**, not less: the benefit scales with participant count — instead of overlapping 3 round trips, pipelining can now overlap up to 9, since the coordinator dispatches intents to all participant tablets concurrently rather than waiting tablet-by-tablet
- still one commit: index maintenance is fully transactional, not an eventual/side write — the single commit record on the transaction status tablet atomically finalizes the base-row intents and both indexes' intents together, so a reader can never observe the row without its index entries (or vice versa), no matter how many tablets were involved

(reads ?) collocated indexes, key-only retrieval, covering indexes, index ordered tables

- file access method-
 - sequential scan; has every column you need
 - index (a subset of columns) retrieval; use the index to identify the rows you need, then go back to the table proper to get any remaining columns, 2 logical reads
- collocated (tables) indexes
 - (yugabyteDB has collocated tables, a different topic, and not covered here)
 - Sybase 1987, Oracle 1997, Spanner 2012, but all meant slightly different things
 - topically clustered indexes/tables reside in this neighborhood
 - "attached index", index follows partitioning strategy of table proper
 - largely; create physical adjacency of parent and child (rows), reduce latency to retrieve
- the biggest downside is creating node hot spots

...

(reads ?) collocated indexes, key-only retrieval, covering indexes, index ordered tables

...

- yugabyteDB index ordered tables
 - data in table exists in key order, no separate heap layer
 - primary key lookup; index ordered data, and all data is present
- key only retrieval
 - column list columns all found in index structure, no need to access table proper
 - can be an exception to index negation

...

example: key only retrieval, index negation exception

```
-- 200,000 rows, bucket uniformly 0-99 (so `bucket < 40` matches ~40% – poor selectivity)
CREATE TABLE t1 (
  id          INT,
  bucket      INT,
  payload     TEXT,
  PRIMARY KEY (id HASH)
);

CREATE INDEX t1_idx ON t1 (bucket ASC);

SELECT id, bucket FROM t1 WHERE bucket < 40;
-- Seq Scan on t1   (actual time=2.044..138.039 rows=80000)
--   Storage Filter: (bucket < 40)

SELECT bucket FROM t1 WHERE bucket < 40;
-- Index Only Scan using t1_idx  (actual time=0.923..19.420 rows=80000)
--   Index Cond: (bucket < 40)
--   Heap Fetches: 0
```

(reads ?) collocated indexes, key-only retrieval, covering indexes, index ordered tables

...

- covering indexes
 - available for both unique and duplicate indexes (more commonly used for unique)
 - additional columns not part of (key calculation), but stored as real, directly-readable column data
- why? pick up key-only retrievals (performance)

example: covering index

```
CREATE TABLE t1 (          -- add 200k rows of data
  id          INT,
  bucket      INT,
  payload     TEXT,
  PRIMARY KEY (id HASH)
);
```

```
EXPLAIN (ANALYZE) SELECT id, bucket FROM t1 WHERE bucket < 40;
-- Seq Scan on t1  (actual time=2.185..139.998 rows=80000 loops=1)
--   Storage Filter: (bucket < 40)
-- Execution Time: 154.017 ms
```

```
CREATE INDEX t1_bucket_idx ON t1 (bucket ASC) INCLUDE (id);
ANALYZE t1;
```

```
EXPLAIN (ANALYZE) SELECT id, bucket FROM t1 WHERE bucket < 40;
-- Index Only Scan using t1_bucket_idx on t1  (actual time=1.865..27.611 rows=80000 loops=1)
--   Index Cond: (bucket < 40)
--   Heap Fetches: 0
-- Execution Time: 44.605 ms
```

example 04



difficulty

new topics, and reinforcing what we have already covered
two table, outer join

example 04:

```
SELECT *  
FROM t1  
LEFT OUTER JOIN t2  
  ON t1.col1 = t2.col1;
```

5 rules applied in order to
determine query plan

- outer joins
- joins
- filters
- table size
- table cardinality

query plan

- file (table) access method
- join method
- *table order*

outer join, what is it

- one table is dominant, another inner/preserved/null-supplying
- if no join pair from the inner table, populate those columns with NULLs
 - every row from the outer table qualifies the join condition
- examples
 - customer → order even if the customer never placed an order
 - employee → department even for weird employee types (contractors ?) without a department
- consider how to process if t1 is read first, versus t2

example 04: historically (and without exception)

```
SELECT *  
FROM t1  
LEFT OUTER JOIN t2 ON t1.col1 = t2.col1;
```



- always-always-always read the outer table first
 - if you read the inner table first, you would have to go back and figure out what you are missing from the outer table. blecht
- nested-loop-join (NLJ) down into the inner table
- comparably super CPU and I/O intensive
- at least we could batch nested loop join (BNLJ) to reduce network I/O, I/O calls

example 04

solution

example 04: the actual code behind this

```
CREATE TABLE t1 (  
    col1 INT,  
    col2 INT,  
    PRIMARY KEY (col1 HASH)  
);  
INSERT INTO t1 (col1, col2)  
SELECT g, g * 2  
FROM generate_series(1, 100000) AS g;  
  
CREATE TABLE t2 (  
    col1 INT,  
    col2 INT,  
    PRIMARY KEY (col1 HASH)  
);  
INSERT INTO t2 (col1, col2)  
SELECT g, g * 3  
FROM generate_series(1, 10000) AS g;  
  
ANALYZE t1;  
ANALYZE t2;
```

example 04: the actual code behind this

```
-- QUERY PLAN
-- -----
-- Hash Left Join  (cost=1002.61..9767.79 rows=100000 width=16)
--   (actual time=27.894..145.244 rows=100000 loops=1)
--   Hash Cond: (t1.col1 = t2.col1)
--   -> Seq Scan on t1  (cost=21.09..8523.76 rows=100000 width=8)
--     (actual time=3.513..84.543 rows=100000 loops=1)
--   -> Hash  (cost=856.52..856.52 rows=10000 width=8) (actual time=24.310..24.311 rows=10000 loops=1)
--     Buckets: 16384  Batches: 1  Memory Usage: 519kB
--     -> Seq Scan on t2  (cost=21.08..856.52 rows=10000 width=8)
--       (actual time=1.677..21.353 rows=10000 loops=1)
-- Planning Time: 79.297 ms
-- Execution Time: 167.949 ms
-- Peak Memory Usage: 923 kB
```

3 sequentially scan t1

2 build a hash list of that result

1 sequentially scan t2



example 04: the actual code behind this

- 1 sequentially scan t2
 - why ?
 - there is no filter/predicate, you are examining every row anyway
- 2 build a hash list of that result from t2
 - why ? (there was already a hash index in place, why not use that)
 - these rows have to move anyway. they have to meet/join with tuples from t1
 - these rows will all move to a single node, in memory (work_mem)
 - why ?
 - the choices are;
 - nested-loop-join, move a little at a time, small targeted fetches
 - hash-join, move all at once
- 3 sequentially scan t1
 - why ? (same answer as above; no filter predicate)
 - why not also move this table ?
 - there is no reason to. we only examine this table one time

outer joins

- still a major influence on the execution of the query
 - it is the first rule of the optimizer for a reason
 - (without a predicate, every row from the dominant table gets returned)
- give the optimizer more options-
 - model the (missing department) as data, as a relation
 - the \$0 first customer order

example 04: what if we disable hash join

```
SET enable_hashjoin = off;           -- not an 'optimizer hint', actually a 'session level setting'
SET enable_mergejoin = off;

EXPLAIN (ANALYZE)
SELECT *
FROM t1
LEFT OUTER JOIN t2 ON t1.col1 = t2.col1;
```

example 04: what if we disable hash join

QUERY PLAN

```
YB Batched Nested Loop Left Join (cost=42.17..30799.29 rows=100000 width=16)
  (actual time=4.415..304.477 rows=100000 loops=1)
  Join Filter: (t1.col1 = t2.col1)
  -> Seq Scan on t1 (cost=21.09..8523.76 rows=100000 width=8)
      (actual time=1.013..16.275 rows=100000 loops=1)
  -> Index Scan using t2_pkey on t2 (cost=21.08..215.30 rows=1024 width=8)
      (actual time=2.314..2.331 rows=102 loops=98)
      Index Cond: (col1 = ANY (ARRAY[t1.col1, $1, $2, ..., $1023]))
```

Planning Time: 59.343 ms

Execution Time: 321.912 ms

Peak Memory Usage: 1779 kB

```
-- with hash join, the first plan
--
-- Planning Time: 79.297 ms
-- Execution Time: 167.949 ms
-- Peak Memory Usage: 923 kB
```


example 04: what if there were filters

```
EXPLAIN (ANALYZE)
SELECT *
FROM t1
LEFT OUTER JOIN t2 ON t1.col1 = t2.col1
WHERE t1.col1 <= 100;
```

example 04: what if there were filters (a different query)

QUERY PLAN

```
YB Batched Nested Loop Left Join (cost=42.17..6098.70 rows=104 width=16)
  (actual time=35.292..35.344 rows=100 loops=1)
  Join Filter: (t1.col1 = t2.col1)
  -> Seq Scan on t1 (cost=21.09..6057.84 rows=104 width=8)
    (actual time=33.530..33.539 rows=100 loops=1)
    Storage Filter: (col1 <= 100)
  -> Index Scan using t2_pkey on t2 (cost=21.08..40.73 rows=104 width=8)
    (actual time=1.672..1.684 rows=100 loops=1)
    Index Cond: (col1 = ANY (ARRAY[t1.col1, $1, $2, ..., $1023]))
Planning Time: 77.439 ms
Execution Time: 36.627 ms
Peak Memory Usage: 659 kB
```

```
-- with hash join, the first plan
--
-- Planning Time: 79.297 ms
-- Execution Time: 167.949 ms
-- Peak Memory Usage: 923 kB
```

```
-- BNLJ, the second plan
--
-- Planning Time: 59.343 ms
-- Execution Time: 321.912 ms
-- Peak Memory Usage: 1779 kB
```

checkpoint

- why are we less concerned about having 8 indexes on one table than we were umpteen years ago
- write pipelining is trying to improve upon (pick one); sla delivery time, cpu utilization, ram utilization, network utilization, disk space, data consistency
- [what type of index retrieval might overcome an index negation](#)
- why are you more likely to use covering indexes for unique indexes versus duplicate indexes
- two table outer joined select; all things being equal, which table is returning all rows
- how might you model away from outer joins
- how does batch nested loop join differ from nested loop join
- which resource are we trying to optimize here; network, cpu, ram, disk
- we've witnessed a few 'session level settings'. name one

example 04

hands on exercise

- exercise 04

example 05

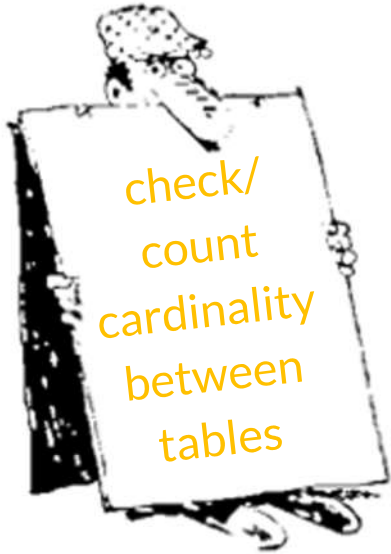


difficulty

the most costly single word modifier to any query
two tables, and a/the "select modifier"

example 05

```
SELECT DISTINCT t1.col1, t1.col4, t1.col9, t1.col10, t2.status
FROM t1
JOIN t2 ON t1.col1 = t2.col1
WHERE t2.status = 'active';
```



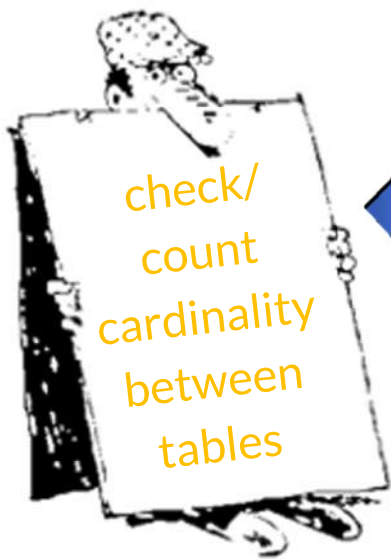
5 rules applied in order to determine query plan

- outer joins
- joins
- filters
- table size
- table cardinality

query plan

- file (table) access method
- join method
- *table order*

an aside: counting, and knowing what is going on-



we just said

but also

joining two tables, creates a projection, a virtual what-if result set

- a 1M row table joined with a 1M row table, creates a percentage of 1,000 billion/B row intermediate result, (percent of 1 trillion/T)
- when both tables have filters, you might never see this result set. you feel it (slow query, busy CPU), but you never see it. you never return it to the client

you don't know if you do not count

we say FJS (filter → join → sort)

but really it can be

filter → join → filter → join → filter ...

logically, how SELECT DISTINCT gets processed

- process the query to get a result set
 - we need to do this first because a result set might come from multiple tables, or derived columns; the result set isn't available to process until it exists
- **sort on the value of every column in the column list** (blecht)
 - sorts-
 - all sorts are blocking operations, a total data set operation. (blocking because the last tuple produced might sort first)
 - distributed systems; you can push (sub-sorts) to individual nodes, but, you then have either one of two choices-
 - broadcast, re-distribute these sub-sorted results to complete (processing)
 - push the sub-sorts to a single node for the final (processing)
- merge
- remove adjacent duplicate entries

example 05: the actual code behind this

```
CREATE TABLE t1 (  
  col1    INT,  
  col2    INT,  
  col3    INT,  
  col4    TEXT,  
  col5    TEXT,  
  col6    INT,  
  col7    TIMESTAMP,  
  col8    NUMERIC,  
  col9    TEXT,  
  col10   TEXT,  
  PRIMARY KEY (col1 HASH)  
);
```

example 05: the actual code behind this

```
INSERT INTO t1 (col1, col2, col3, col4, col5, col6, col7, col8, col9, col10)
SELECT
    g,
    g % 1000,
    g % 50,
    'name_' || g,
    'category_' || (g % 20),
    g % 7,
    TIMESTAMP '2026-01-01' + (g % 365) * INTERVAL '1 day',
    (g % 10000) / 100.0,
    'ref_' || (g % 500),
    repeat('The quick brown fox jumps over the lazy dog. ', 5) || 'entity #' || g
FROM generate_series(1, 100000) AS g;
```

example 05: the actual code behind this

```
CREATE TABLE t2 (  
  id      BIGINT,  
  coll    INT,  
  status  TEXT,  
  PRIMARY KEY (id HASH)  
);  
  
INSERT INTO t2 (id, coll, status)  
SELECT  
  g,  
  ((g - 1) % 100000) + 1,  
  CASE WHEN g % 3 = 0 THEN 'active' ELSE 'inactive' END  
FROM generate_series(1, 1000000) AS g;  
  
CREATE INDEX t2_coll_idx ON t2 (coll HASH);  
  
ANALYZE t1;  
ANALYZE t2;
```

example 05: the actual code behind this

QUERY PLAN

```
Unique (cost=232668.46..236755.04 rows=200000 width=270)
  (actual time=1570.274..1676.840 rows=100000 loops=1)
    -> Sort (cost=232668.46..233485.77 rows=326927 width=270)
      (actual time=1570.271..1598.808 rows=333333 loops=1)
        Sort Key: t1.col1, t1.col4, t1.col9, t1.col10
        Sort Method: external merge Disk: 92360kB
        -> Hash Join (cost=35561.17..120033.22 rows=326927 width=270)
          (actual time=681.834..1343.021 rows=333333 loops=1)
            Hash Cond: (t2.col1 = t1.col1)
            -> Seq Scan on t2 (cost=21.10..73127.94 rows=339867 width=12)
              (actual time=4.819..474.296 rows=333333 loops=1)
                Storage Filter: (status = 'active'::text)
            -> Hash (cost=30774.07..30774.07 rows=100000 width=262)
              (actual time=676.910..676.911 rows=100000 loops=1)
                Buckets: 32768 Batches: 4 Memory Usage: 7582kB
                -> Seq Scan on t1 (cost=21.09..30774.07 rows=100000 width=262)
                  (actual time=7.532..633.983 rows=100000 loops=1)
```

Planning Time: 55.844 ms

Execution Time: 1701.840 ms

Peak Memory Usage: 15206 kB

example 05

solution

SELECT DISTINCT, why are we doing this

- ad-hoc and exploratory analysis
 - what distinct product categories had sales last quarter
 - occasional or one-off queries no one knew in advance. you can't reasonably model an attribute/table for this
- combining multiple independent sources into one list
- when it is structurally required, not just convenient-
 - when a join crosses a 1:M or M:M relationship, but the business question is about the "one" side only
 - happens infrequently, okay
 - happens often, should this be an attribute in the "one" side, the parent table
 - if you allow triggers; this can be automated. no triggers, then it's business logic
 - *should this computation be paid for at run time, repetitively, or maintained (live)*
- often though, this could be a lazy add to a query that no one fully understood

SELECT DISTINCT, how to find/avoid/repair-

- implied on prior page-
 - if used often, this is "state", an attribute. model as a column in a parent table
 - is this customer in default (payment). if this question happens often, it is state, an attribute
- how to find/avoid/repair-
 - select count tables in the relationship; is the inner table truly a many
 - change the query shape
 - do the outer query, remove duplicates if needed
 - then join to a subquery (not a correlated subquery)
- can this be moved to a smaller set of columns. example (PostgreSQL specific, not ANSI SQL, returns first in group),

```
SELECT DISTINCT ON (location) location, time, report
FROM weather_reports
ORDER BY location, time DESC;
```

checkpoint

- you have a list of 100 queries from your system sorted by run time descending (longest queries at the top of the list); which single, optional keyword appears in at least $\frac{1}{4}$ of the top 10 worst queries

example 05

hands on exercise

- exercise 05

example 06



difficulty

common table expressions

correlated subqueries, **having** to consider cardinality between tables

manual query re-writes to improve query plan performance

common table expressions (CTEs)

SQL standard, SQL:1999

CTEs (the WITH clause) were formally added in the fourth major SQL revision, with ISO standards issued 1999–2002.

first commercial implementation: SQL Server 2005 — Microsoft was the first major vendor to actually ship it, about six years after standardization

PostgreSQL — version 8.4, 2009

MariaDB — version 10.2.1

MySQL — version 8.0, 2018 — notably late, over a decade behind SQL Server

common table expressions (CTEs), why-

readability is the primary benefit for the common case, but there are 3 genuine, real must use CTE scenarios

1. recursive/hierarchical queries — the clearest case. a subquery cannot reference itself. full stop
2. data-modifying CTEs — genuinely unique capability, not just style
PostgreSQL/MySQL let you chain an INSERT/UPDATE/DELETE ... RETURNING into a subsequent query, all in one atomic statement —
e.g., delete expired rows and insert them into an archive table in a single DML statement, rather than two separate statements in a transaction
3. explicit optimizer control via MATERIALIZED / NOT MATERIALIZED — a real capability gap

common table expressions (CTEs), example

```
WITH big_report AS MATERIALIZED (  
    SELECT * FROM orders                -- computes ALL rows here, no filtering yet  
)  
SELECT * FROM big_report WHERE customer_id = 42;
```

big_report computes and stores every row in orders first — a full scan, materialized as a temp result — and only then filters down to customer_id = 42. The customer_id = 42 filter never gets to influence the CTE's own execution

```
WITH big_report AS NOT MATERIALIZED (  
    SELECT * FROM orders  
)  
SELECT * FROM big_report WHERE customer_id = 42;
```

same SQL text, but now the planner is allowed to effectively rewrite this as SELECT * FROM orders WHERE customer_id = 42 from the start — an index seek on customer_id, not a full scan followed by filtering

common table expressions (CTEs), why-

two real reasons to force it, both about control, not raw speed:

- side-effect functions. if our CTE calls something like `nextval()` or a volatile function, NOT MATERIALIZED could let the planner evaluate it more than once (once per outer reference) — MATERIALIZED guarantees it runs exactly once, with a stable, reusable result
- we know better than the planner for this specific query. sometimes the planner's guess at pushing filters in is wrong, and forcing full materialization first is actually

the default, if you specify neither, PostgreSQL/MySQL picks for you, using a simple rule:

referenced more than once, or recursive → materialized

referenced exactly once, non-recursive → not materialized (i.e., eligible for inlining/pushdown)

using CTEs to actually make a query faster, problem 1

```
SELECT c.customer_id, c.name,  
       (SELECT COUNT(*)      FROM orders o WHERE o.customer_id = c.customer_id) AS order_count,  
       (SELECT SUM(o.amount) FROM orders o WHERE o.customer_id = c.customer_id) AS total_spent  
FROM customers c  
WHERE c.region = 'West';
```

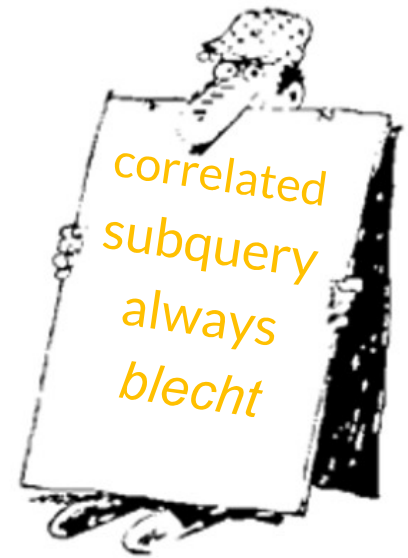
for every single customer row, this fires two separate correlated subquery executions against orders — one to count, one to sum, each independently seeking/scanning for that customer's rows
for 10,000 West-region customers, that's up to 20,000 separate subquery invocations against orders, not one pass over the table



using CTEs to actually make a query faster, problem 1

```
SELECT c.customer_id, c.name,  
       (SELECT COUNT(*)      FROM orders o WHERE o.customer_id = c.customer_id) AS order_count,  
       (SELECT SUM(o.amount) FROM orders o WHERE o.customer_id = c.customer_id) AS total_spent  
FROM customers c  
WHERE c.region = 'West';
```

historically, correlated subqueries are always bad-
before CTEs we tried to rewrite these as joins
the issue is always cardinality; if the (inner) table produces multiple
rows, now we output (n) rows from the parent, not just one row
this then caused us to use DISINCT, already covered
because we have an aggregate here, we produce one inner row



using CTEs to actually make a query faster, solution 1

```
WITH order_stats AS (  
  SELECT customer_id, COUNT(*) AS order_count, SUM(amount) AS total_spent  
  FROM orders  
  GROUP BY customer_id  
)  
SELECT c.customer_id, c.name, os.order_count, os.total_spent  
FROM customers c  
JOIN order_stats os ON os.customer_id = c.customer_id  
WHERE c.region = 'West';
```

orders gets scanned and aggregated exactly once — a single GROUP BY pass producing one summary row per customer_id — and that result is joined to customers. whether we have 100 customers or 100,000, orders is still touched exactly once
the correlated version's cost scales with the number of outer rows; the CTE version's cost scales with the size of the data being aggregated, computed a single time.

common table expressions (CTEs), solution 1, verifying

whether the CTE version is actually faster depends on real factors —

- whether `orders.customer_id` is indexed (the correlated version leans hard on that index existing; without it, each correlated subquery is its own mini table scan, making the slow version catastrophically slow rather than just moderately slow)
- how selective the region filter is

the way to confirm, run `EXPLAIN ANALYZE` on both versions and look for the correlated version showing the subquery plan node repeated inside a loop (nested Loop, subplan re-executed) versus the CTE version showing one `GroupAggregate` or `HashAggregate` node feeding a single join

common table expressions (CTEs), *other repetitive execution patterns*

- 1 same non-correlated subquery pasted in multiple places
- 2 multiple UNION branches each re-filtering the same expensive base
- 3 repeated self-joins to the same huge table for period comparisons
- 4 repeated complex expression case

CTEs, same non-correlated subquery pasted in multiple places, problem

```
SELECT customer_id,  
       (SELECT AVG(amount) FROM orders WHERE order_date >= '2026-01-01') AS company_avg,  
       amount,  
       amount - (SELECT AVG(amount) FROM orders WHERE order_date >= '2026-01-01') AS diff_from_avg  
FROM orders  
WHERE amount > (SELECT AVG(amount) FROM orders WHERE order_date >= '2026-01-01') * 2;
```

same `AVG(amount)` calculation over the same date-filtered orders table appears three separate times — it's not correlated (doesn't depend on the outer row), but many optimizers still won't recognize "these three subqueries are textually identical" and dedupe them automatically



CTEs, same non-correlated subquery pasted in multiple places, solution

```
WITH company_avg AS (  
    SELECT AVG(amount) AS avg_amount FROM orders WHERE order_date >= '2026-01-01'  
)  
SELECT  
    o.customer_id, ca.avg_amount AS company_avg, o.amount,  
    o.amount - ca.avg_amount AS diff_from_avg  
FROM orders o, company_avg ca  
WHERE o.amount > ca.avg_amount * 2;
```

1

CTEs, multiple UNION branches each re-filtering the same expensive base, problem

```
-- slow, the expensive join+filter runs independently in each branch
--
SELECT 'high_value' AS segment, COUNT(*)
FROM orders o JOIN customers c ON c.customer_id = o.customer_id
WHERE c.region = 'West' AND o.amount > 1000
UNION ALL
SELECT 'repeat_buyer', COUNT(*)
FROM orders o JOIN customers c ON c.customer_id = o.customer_id
WHERE c.region = 'West' AND o.order_count > 5;
```

both branches independently redo the same orders JOIN customers WHERE region = 'West' work

2

CTEs, multiple UNION branches each re-filtering the same expensive base, solution

```
WITH west_orders AS (  
  SELECT o.* FROM orders o JOIN customers c ON c.customer_id = o.customer_id  
  WHERE c.region = 'West'  
)  
SELECT 'high_value', COUNT(*) FROM west_orders WHERE amount > 1000  
UNION ALL  
SELECT 'repeat_buyer', COUNT(*) FROM west_orders WHERE order_count > 5;
```

2

CTEs, repeated self-joins to the same huge table for period comparisons, problem

-- Slow: orders gets independently scanned/filtered twice, once per period

--

```
SELECT c.customer_id,  
       SUM(CASE WHEN o1.order_date >= '2026-06-01' THEN o1.amount END) AS this_period,  
       SUM(CASE WHEN o2.order_date >= '2026-01-01' AND o2.order_date < '2026-06-01' THEN o2.amount END) AS last_period  
FROM customers c  
LEFT JOIN orders o1 ON o1.customer_id = c.customer_id AND o1.order_date >= '2026-06-01'  
LEFT JOIN orders o2 ON o2.customer_id = c.customer_id AND o2.order_date >= '2026-01-01' AND o2.order_date < '2026-06-01'  
GROUP BY c.customer_id;
```

orders gets joined to twice, each join independently scanning/filtering the whole table for its own date window

3

CTEs, repeated self-joins to the same huge table for period comparisons, solution

-- pre-aggregate each period once into its own small CTE, then join those small results
-- instead of the raw table:

```
WITH
  this_period AS (
    SELECT customer_id, SUM(amount) AS total FROM orders
    WHERE order_date >= '2026-06-01' GROUP BY customer_id
  ),
  last_period AS (
    SELECT customer_id, SUM(amount) AS total FROM orders
    WHERE order_date >= '2026-01-01' AND order_date < '2026-06-01' GROUP BY customer_id
  )
SELECT c.customer_id, tp.total AS this_period, lp.total AS last_period
FROM customers c
LEFT JOIN this_period tp ON tp.customer_id = c.customer_id
LEFT JOIN last_period lp ON lp.customer_id = c.customer_id;
```

3

CTEs, repeated complex expression case, problem (vector search example)

```
SELECT id, content, embedding <=> '[0.12, 0.05, ...]' AS distance
FROM documents
WHERE embedding <=> '[0.12, 0.05, ...]' < 0.3
ORDER BY embedding <=> '[0.12, 0.05, ...]'
LIMIT 10;
```

that `embedding <=> '[...]'` (cosine/L2 distance against your query vector) shows up once in `SELECT`, once in `WHERE`, once in `ORDER BY`
unlike a cheap predicate like `region = 'West'`, computing vector distance is real, non-trivial per-row math across every dimension of the embedding
repeating it three times per row roughly triples that cost, and Postgres/MySQL doesn't automatically recognize "these three expressions are textually identical, compute once" the way you might assume — each clause gets evaluated independently

4

CTEs, repeated complex expression case, solution

```
WITH scored AS (  
  SELECT id, content, embedding <=> '[0.12, 0.05, ...]' AS distance  
  FROM documents  
)  
SELECT id, content, distance  
FROM scored  
WHERE distance < 0.3  
ORDER BY distance  
LIMIT 10;
```

the expensive vector-distance calculation runs once per row, gets a name (distance), and everything downstream just references that already-computed value

4

example 06: the actual code behind this

```
-- t1 = the "one side" table (e.g. customers), 20,000 rows
CREATE TABLE t1 (
    col1 INT PRIMARY KEY,      -- entity id
    col2 TEXT                  -- category/region, used as a filter
);

-- t2 = the "many side" table (e.g. orders), 100,000 rows
CREATE TABLE t2 (
    col1 INT PRIMARY KEY,      -- row id
    col2 INT,                  -- foreign key back to t1.col1
    col3 NUMERIC(10,2)         -- a numeric value to aggregate
);
```

example 06: the actual code behind this

```
CREATE INDEX idx_t2_col2 ON t2 (col2 ASC);

-- 20,000 rows into t1, evenly split across 4 categories
INSERT INTO t1 (col1, col2)
SELECT g,
       CASE WHEN g % 4 = 0 THEN 'West'
            WHEN g % 4 = 1 THEN 'East'
            WHEN g % 4 = 2 THEN 'North'
            ELSE 'South' END
FROM generate_series(1, 20000) g;
```

example 06: the actual code behind this

```
-- 100,000 rows into t2, ~5 rows per t1 entity on average
INSERT INTO t2 (col1, col2, col3)
SELECT g, ((g % 20000) + 1), (random() * 500)::numeric(10,2)
FROM generate_series(1, 100000) g;

ANALYZE t1;
ANALYZE t2;
```

example 06:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT t1.col1, t1.col2,
       (SELECT COUNT(*) FROM t2      WHERE t2.col2 = t1.col1) AS row_count,
       (SELECT SUM(t2.col3) FROM t2 WHERE t2.col2 = t1.col1) AS total_val
FROM t1
WHERE t1.col2 = 'West';
```

5 rules applied in order to
determine query plan

- outer joins
- joins
- filters
- table size
- table cardinality

query plan

- file (table) access method
- join method
- *table order*

example 06: how does this query execute

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT t1.col1, t1.col2,
       (SELECT COUNT(*) FROM t2
        WHERE t2.col2 = t1.col1) AS row_count,
       (SELECT SUM(t2.col3) FROM t2
        WHERE t2.col2 = t1.col1) AS total_val
FROM t1
WHERE t1.col2 = 'West';
```



- how many tables do we have
- what relationship do the tables have (we did discuss this)
- what is the cardinality between the tables
- what is the access method per table
- which table is read first
- generally, what performance do we expect
- what manual re-write should we consider and why
 - *improve readability/maintenance*
 - *and ...*
- are we missing an index ? what query would we run to check if should create that index

example 06

solution

example 06:

QUERY PLAN

Seq Scan on t1 (cost=21.09..370083.31 rows=5000 width=49)
(actual time=5.216..16155.093 rows=5000 loops=1)
Storage Filter: (col2 = 'West'::text)
SubPlan 1
-> Finalize Aggregate (cost=24.42..24.43 rows=1 width=8)
(actual time=1.118..1.119 rows=1 loops=5000)
-> Index Only Scan using idx_t2_col2 on t2 (cost=24.09..24.41 rows=5 width=0)
(actual time=1.095..1.098 rows=1 loops=5000)
Index Cond: (col2 = t1.col1)
Heap Fetches: 0
Partial Aggregate: true
SubPlan 2
-> Aggregate (cost=49.30..49.31 rows=1 width=32)
(actual time=2.105..2.105 rows=1 loops=5000)
-> Index Scan using idx_t2_col2 on t2 t2_1 (cost=48.18..49.28 rows=5 width=6)
(actual time=2.077..2.082 rows=5 loops=5000)
Index Cond: (col2 = t1.col1)
Planning Time: 18.111 ms
Execution Time: 16159.895 ms
Peak Memory Usage: 717 kB

SubPlan — a correlated (or sometimes uncorrelated) subquery executed as its own mini-plan, potentially many times, one per outer row. this is what we have

not able to batch nested loop join across SubPlan boundaries
and, when distributed, these are RPC calls/costs, not just a cached, local result

InitPlan (not shown) — the uncorrelated special case: a subquery with no reference to the outer row, so it only needs to run once total (its result is a constant reused for every outer row)

example 06b: manual rewrite

```
EXPLAIN (ANALYZE, BUFFERS)
WITH agg AS (
    SELECT col2 AS entity_id, COUNT(*) AS row_count, SUM(col3) AS total_val
    FROM t2
    GROUP BY col2
)
SELECT t1.col1, t1.col2, agg.row_count, agg.total_val
FROM t1
JOIN agg ON agg.entity_id = t1.col1
WHERE t1.col2 = 'West';
```

example 06b: manual rewrite

QUERY PLAN

```
Hash Join (cost=11498.26..12110.88 rows=4991 width=49) (actual time=150.750..164.493 rows=5000 loops=1)
  Hash Cond: (t2.col2 = t1.col1)
  Buffers: temp read=60 written=129
  -> HashAggregate (cost=10054.70..10304.24 rows=19963 width=44) (actual time=144.144..152.721 rows=20000 loops=1)
    Group Key: t2.col2
    Batches: 5 Memory Usage: 8241kB Disk Usage: 696kB
    Buffers: temp read=60 written=129
    -> Seq Scan on t2 (cost=21.09..9304.70 rows=100000 width=10) (actual time=1.407..93.302 rows=100000 loops=1)
  -> Hash (cost=1381.05..1381.05 rows=5000 width=9) (actual time=6.570..6.571 rows=5000 loops=1)
    Buckets: 8192 Batches: 1 Memory Usage: 265kB
    -> Seq Scan on t1 (cost=21.09..1381.05 rows=5000 width=9) (actual time=1.281..5.344 rows=5000 loops=1)
      Storage Filter: (col2 = 'West'::text)
Planning Time: 17.392 ms
Execution Time: 165.904 ms
Peak Memory Usage: 8932 kB
```

-- the first plan
Planning Time: 18.111 ms
Execution Time: 16159.895 ms

example 06: what else can be done



example 06c: what else can be done

tuning	result	verdict
SET work_mem = '64MB'	~137ms, Batches: 1 (no disk spill)	modest win (~17%), free/no schema change
filter t2 via join before aggregating (CTE pre-filters to West's 5,000 entities, then aggregates only their rows)	~142ms	wash — the batched-nested-loop probe into the index to pre-filter costs about what it saves in aggregation. Not worth the added complexity here.
covering index: CREATE INDEX idx_t2_col2_covering ON t2 (col2 ASC) INCLUDE (col3)	~73ms	clear winner — ~2.3x over baseline

checkpoint

- CTEs (why)
 - *readability*, (surface nesting, other)
 - *required for*-
 - recursion/hierarchical
 - data modifying
 - explicit control
 - *reduce repetition* (optimization, one-pass versus n probes)
 - same subquery in multiple places
 - multiple union branches with filter/re-filter
 - repetitive self joins
 - repetitive complex expressions
- non-correlated/correlated subqueries
- count, understand operations inside query execution; filters, cardinality between joined tables

example 06

hands on exercise

- exercise 06b

discussion

rules based optimizers

- the last two rules

cost based optimizers (CBO)

- the CBO pipeline
- what actually gets compared
- what statistics actually contain

query optimizer hints

rules based optimizers: the last two rules

```
select *  
from t1, t2  
where  
    t1.col1 = t2.col1  
and  
    t1.col1 = 5  
and  
    t2.col2 > 5  
order by  
    t2.col2;
```

5 rules applied in order to determine query plan

- outer joins
- joins
- filters
- **table size** (a statistic, worst table first)
- **table cardinality**

query plan

- file (table) access method
- join method
- *table order*

the cost based optimizer (CBO) pipeline

- 1.) parse & rewrite — normalize the query, flatten eligible subqueries/views into the main plan
- 2.) generate candidate plans — different access paths, join orders, join algorithms, aggregation strategies
- 3.) estimate cardinality for every sub-step, using table/column statistics
- 4.) apply cost formulas — I/O cost (sequential vs. random page reads) + CPU cost (per-tuple processing)
- 5.) compare total costs across candidates
- 6.) select the minimum-cost plan and execute (or cache it for reuse)

what actually gets compared

- access path per table: sequential scan, index scan, index-only scan, bitmap scan
- join order: which table drives, which is probed — grows combinatorially as tables are added (3+ tables → real search problem, not a lookup)
- join algorithm: nested loop, batched nested loop, hash join, merge join
- aggregation strategy: hash aggregate vs. sorted/group aggregate
- materialization choices: inline a CTE vs. materialize it; flatten a subquery into a join vs. keep it as a SubPlan

search space explosion is why query planners fall back to heuristics/genetic search past a certain table count (e.g., Postgres's GEQO)

what statistics actually contain (table and column level)

table-level:

- estimated row count, disk page count

per column (Postgres/MySQL pg_stats):

- null_frac — fraction of NULLs
- avg_width — average stored byte size
- n_distinct — distinct-value estimate
- most_common_vals / most_common_freqs — explicit top-N values and their exact frequencies (the MCV list)
- histogram_bounds — equi-depth buckets covering everything not in the MCV list
- correlation — how closely physical storage order matches sorted order (-1 to 1); affects the cost of range/ORDER BY scans

what statistics actually contain (beyond single columns)

- extended/multivariate statistics (CREATE STATISTICS) — fixes the default “columns are independent” assumption:
 - combined n_distinct across a column group
 - functional-dependency strength between correlated columns
 - multivariate MCV lists
- index-level stats — index size, distinct key values under that index
- freshness matters as much as accuracy: gathered by ANALYZE (manual or autovacuum-triggered); sample size is configurable — stale or under-sampled stats produce bad cardinality estimates regardless of how good the cost formulas are

atop distributed

- classic statistics answer [how many rows]. a distributed cost model also needs: how many tablets does this plan touch, and what does each network round-trip cost
- batching changes the arithmetic: a batched nested loop join folds up to ~1,024 join keys into a single RPC instead of one-per-row — same underlying statistics, very different real cost
- Proof from our previous examples, same cluster, same table stats:
 - exercise 06 (correlated subqueries, one RPC-equivalent probe per row): ~16.2s
 - exercise 06b (rewritten as one GROUP BY pass): ~166ms
 - same optimizer, same statistics on t1/t2 — the ~100x gap came entirely from how many round trips each query shape required

on a distributed database, query shape interacts with the cost model as strongly as the statistics do

query optimizer hints

- a directive embedded in or near a SQL statement that overrides the query optimizer's own cost-based choice
- changes how the engine gets the answer, never what the answer is
- exists because the optimizer's statistics or cost model can be wrong, stale, or simply absent for an edge case
- treated by most vendors as a last resort, not a first-line tuning tool

what optimizer hints typically control

- table join order — which table drives the query, which gets probed
- join algorithm — nested loop vs. hash join vs. merge join
- access path per file/table — sequential scan vs. index scan vs. index-only scan vs. bitmap scan
- occasionally: parallelism degree, materialization points, or manual row/cost estimate overrides

two ways to hint in PostgreSQL/yugabyteDB

- `pg_hint_plan` extension — structured hints inside a `/*+ ... */` comment placed directly above the query
- session-level GUCs — `SET enable_hashjoin = off`; etc. — blunter, disables a whole strategy class for the entire session
- the two combine well: GUCs remove entire categories of alternative plans, hints pin the exact remaining choice
- both require the hinting mechanism to be installed/enabled server-side (`pg_hint_plan` must be in `shared_preload_libraries`)

query optimizer hints: good use, bad use

good use

- reproducing a historical/vendor-reported plan; a short-term stopgap while the real fix (statistics, indexing) is rolled out
- working around a known, narrow optimizer blind spot

bad use:

- a permanent substitute for accurate statistics or proper index design

the risk:

- a hint that's correct today can force a bad plan tomorrow once data volume or distribution shifts — the optimizer would have adapted on its own; a hint won't

query optimizer hints example

```
SET enable_hashjoin = off;  
SET enable_mergejoin = off;  
SET enable_seqscan = off;
```

```
/*+  
  Leading(((t1 t2) t3))  
  NestLoop(t1 t2)  
  NestLoop(t1 t2 t3)  
  IndexScan(t1 idx_t1_col1_col2)  
  IndexScan(t2 idx_t2_col1_col2_col3)  
  IndexScan(t3 t3_pkey)  
*/  
EXPLAIN (ANALYZE, BUFFERS)  
SELECT t1.col1, t1.col2, t2.col3, t3.col1  
FROM t1, t2, t3  
WHERE t1.col1 = 'LA'  
      AND t1.col2 = t2.col1  
      AND t2.col3 = 'FLT-00025'  
      AND t2.col3 = t3.col1;
```

example 07



difficulty

graduation; the final example
bringing it all together

example 07: graduation, the "upside down filter problem"

- originally-
 - 5 minute query down to subsecond (was rules based)
 - now, cost based, the query optimizer knows; no chance, do this differently, but still slow
- classic pattern, happens often with ISV/COTS software, reinforces all we have discussed
- and reinforces how cost based is much better
- we use hints to show it totally broken
 - *how would you observe this, look to improve*
- then run pure cost based
- then run even more highly optimized run time (better indexes)

example 07: the code behind this

```
CREATE TABLE t1 (  
    col1 TEXT,  
    col2 TEXT,  
    col3 INT,  
    col4 TEXT,  
    col5 TEXT,  
    col6 TIMESTAMP,  
    col7 NUMERIC(10,2),  
    col8 TEXT,  
    PRIMARY KEY (col1 HASH, col2 ASC)  
);
```

example 07: the code behind this

```
CREATE TABLE t2 (  
    id      BIGINT,  
    col1    TEXT,  
    col2    SMALLINT,  
    col3    TEXT,  
    PRIMARY KEY (id HASH)  
);
```

```
CREATE INDEX idx_t2_col1_col2_col3 ON t2 (col1 HASH, col2 ASC, col3 ASC);
```


example 07: the code behind this

```
CREATE TABLE t3 (  
    col1 TEXT,  
    col2 TEXT,  
    col3 INT,  
    col4 TEXT,  
    col5 TEXT,  
    col6 TIMESTAMP,  
    col7 NUMERIC(10,2),  
    col8 TEXT,  
    PRIMARY KEY (col1 HASH)  
);
```

example 07:

```
SELECT t1.col1, t1.col2, t2.col3, t3.col1
FROM t1, t2, t3
WHERE
    t1.col1 = 'LA'
AND
    t1.col2 = t2.col1
AND
    t2.col3 = 'FLT-00025'
AND
    t2.col3 = t3.col1;
```

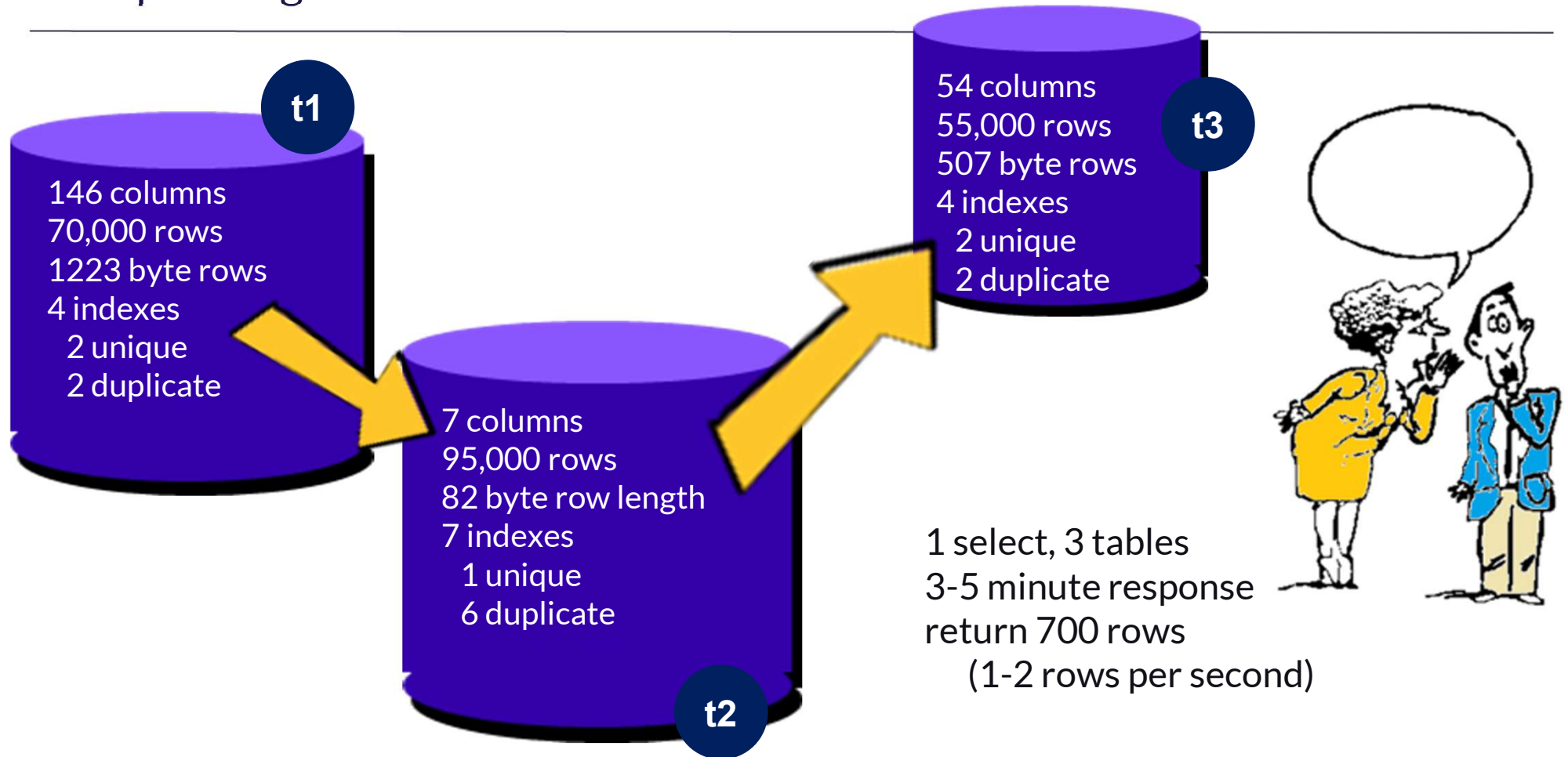
5 rules applied in order to
determine query plan

- outer joins
- joins
- filters
- table size
- table cardinality

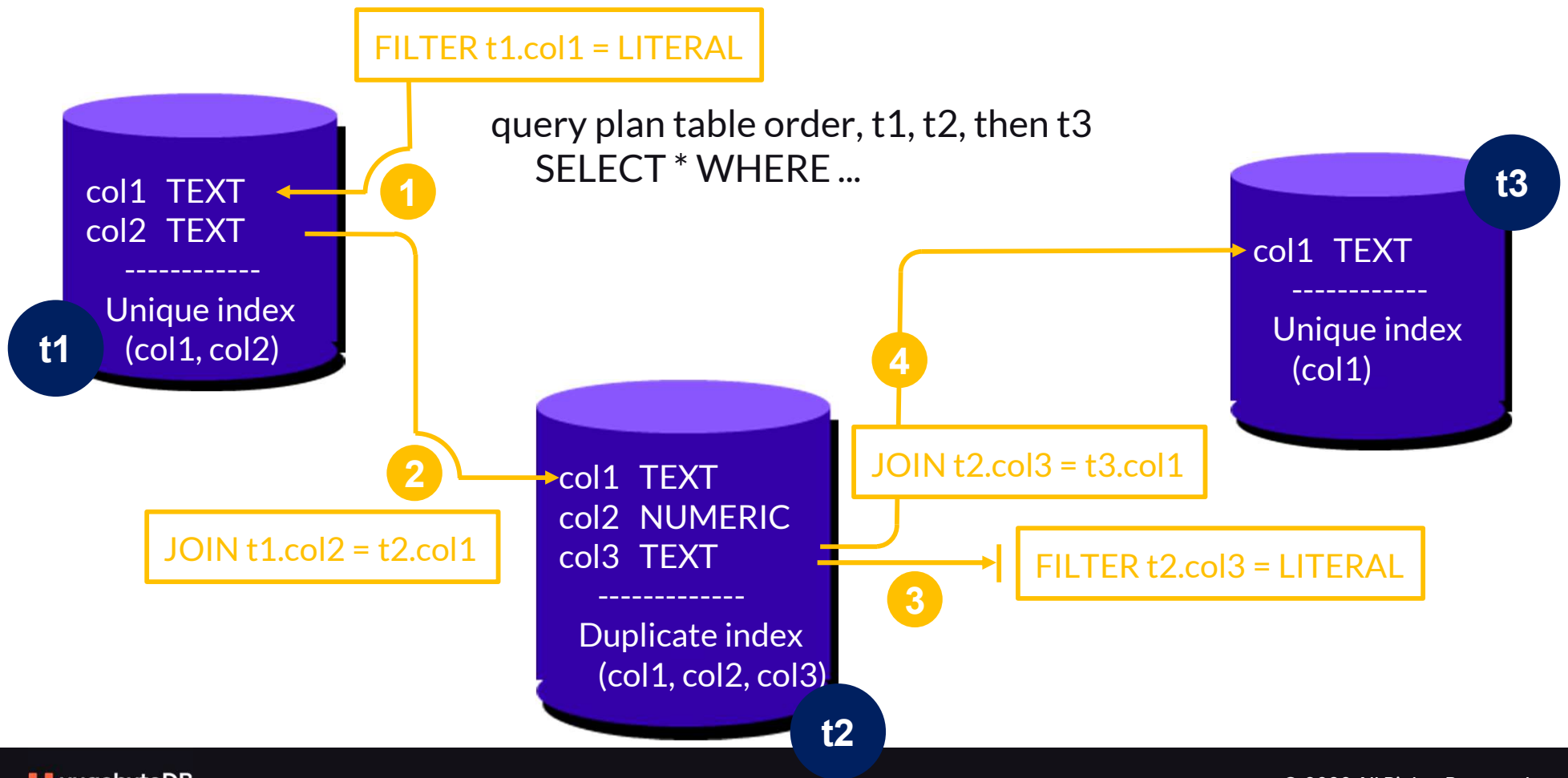
query plan

- file (table) access method
- join method
- *table order*

example 07: gather information



example 07: information, explain



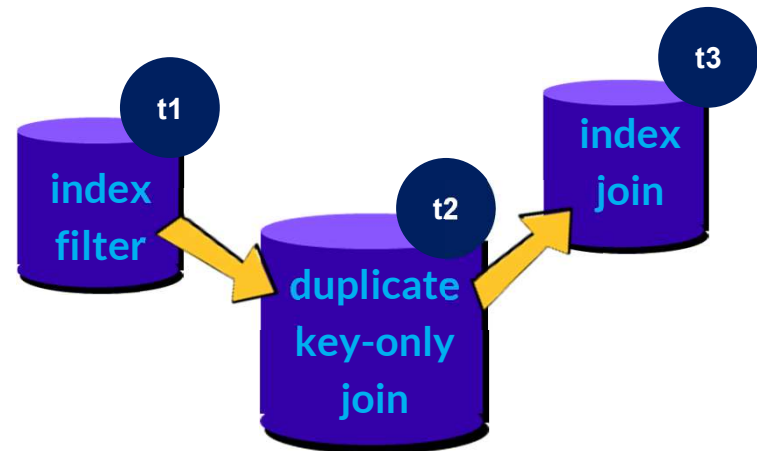
example 07: discussion

analyze did seem to indicate an efficient query plan

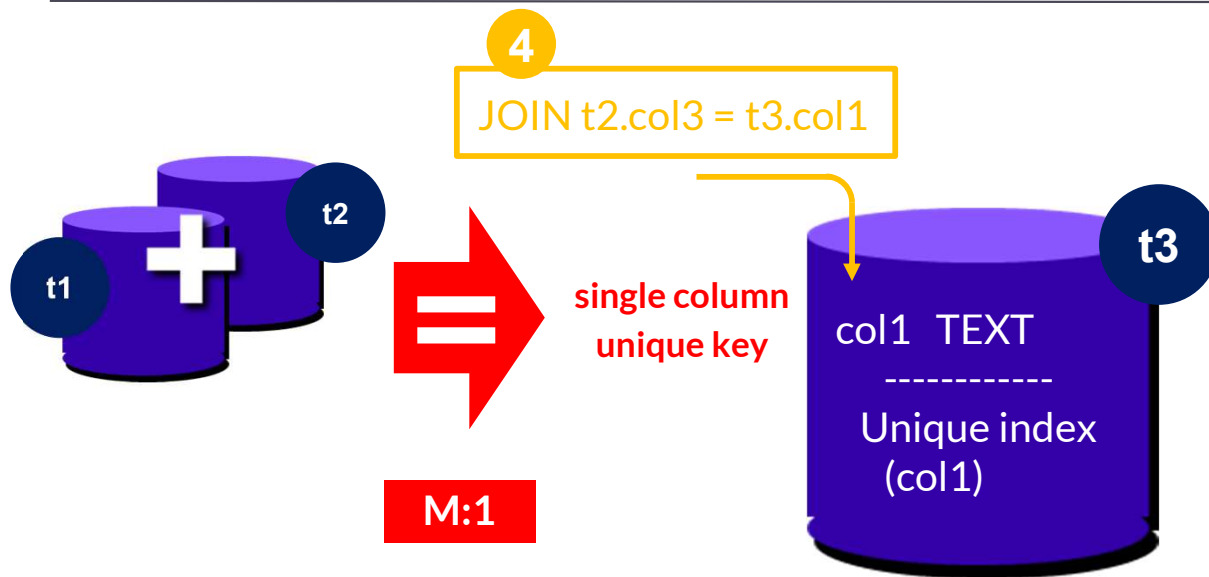
index → index → index

no temporary tables / temporary files

yet the SQL SELECT returned only 700 rows
in three to five minutes

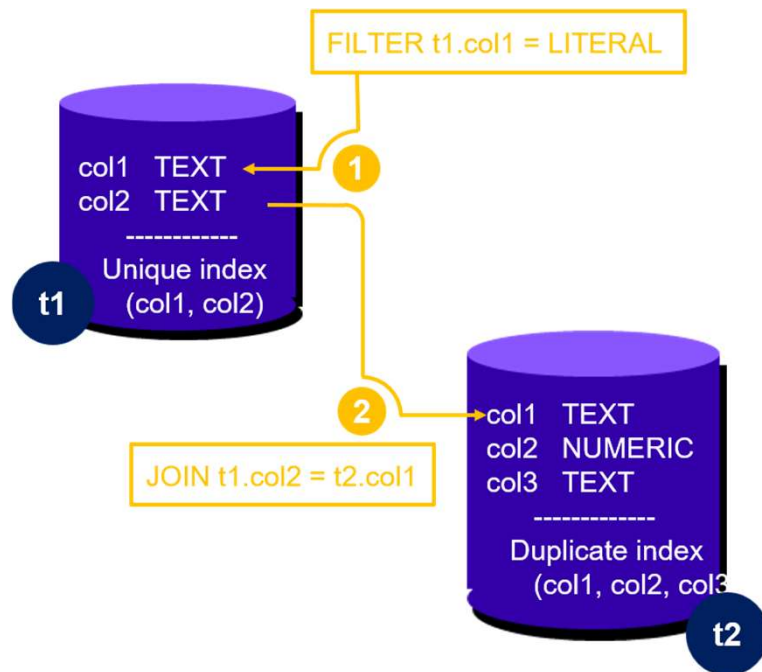


example 07: simplify the test case



- drop table t3 from the query
- as expected, dropping table t3 from the query did not measurably affect the performance of the query

example 07: examine just tables t1 and t2



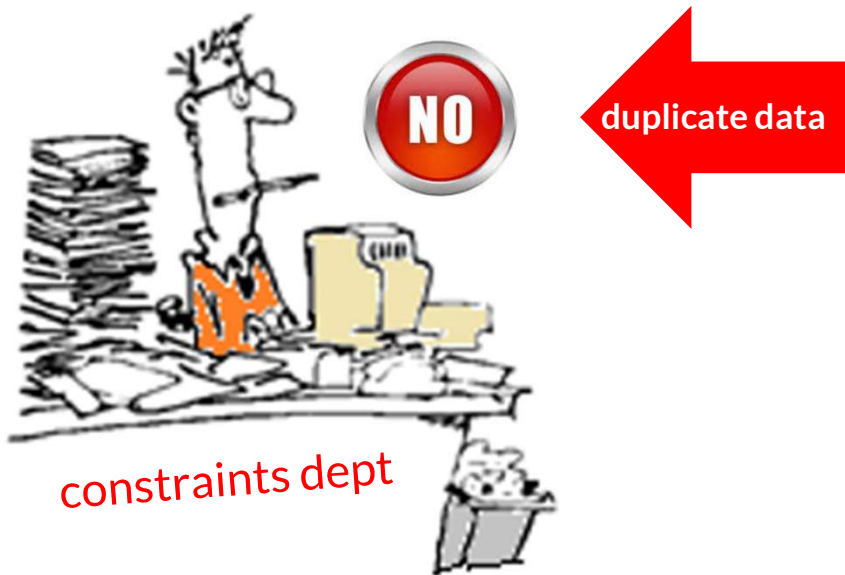
- examine the performance of tables t1 and t2
- examine the tables separately, and then together

example 07: measure selectivity of filters



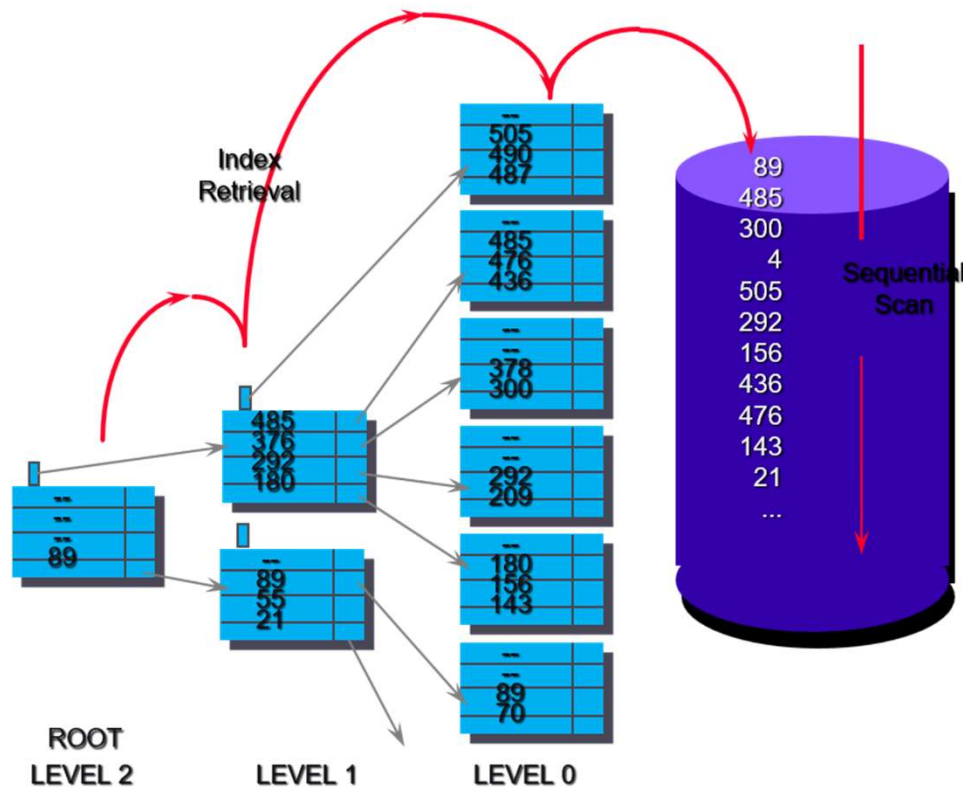
- measure the selectivity of the filter on table t1. test performance on table t1
- this was a canned application system. every data item in the database seemed to lead with a code for location. this customer only had one location

about unique constraints



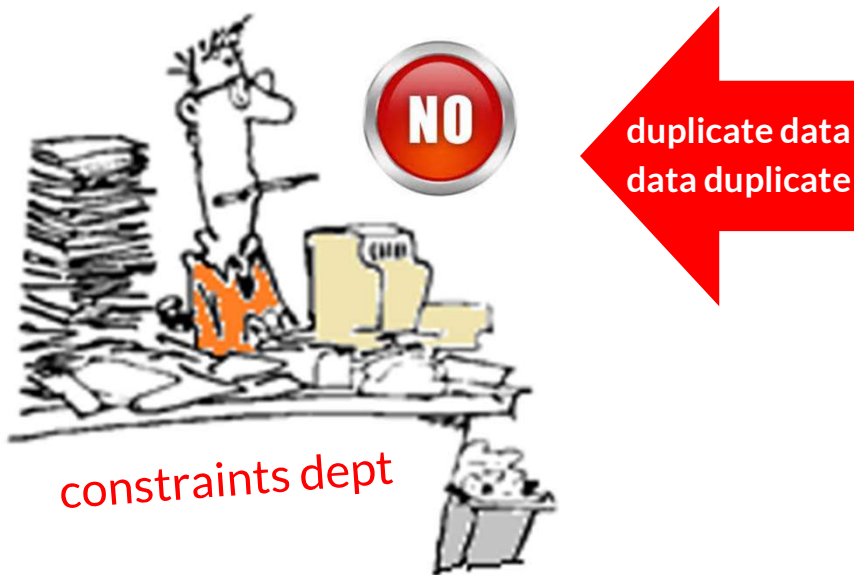
- the unique index on table t1 was required from a business rule stand point
- it was a unique index and provided a unique constraint

legacy systems: index versus sequential access method (B-tree+ shown)



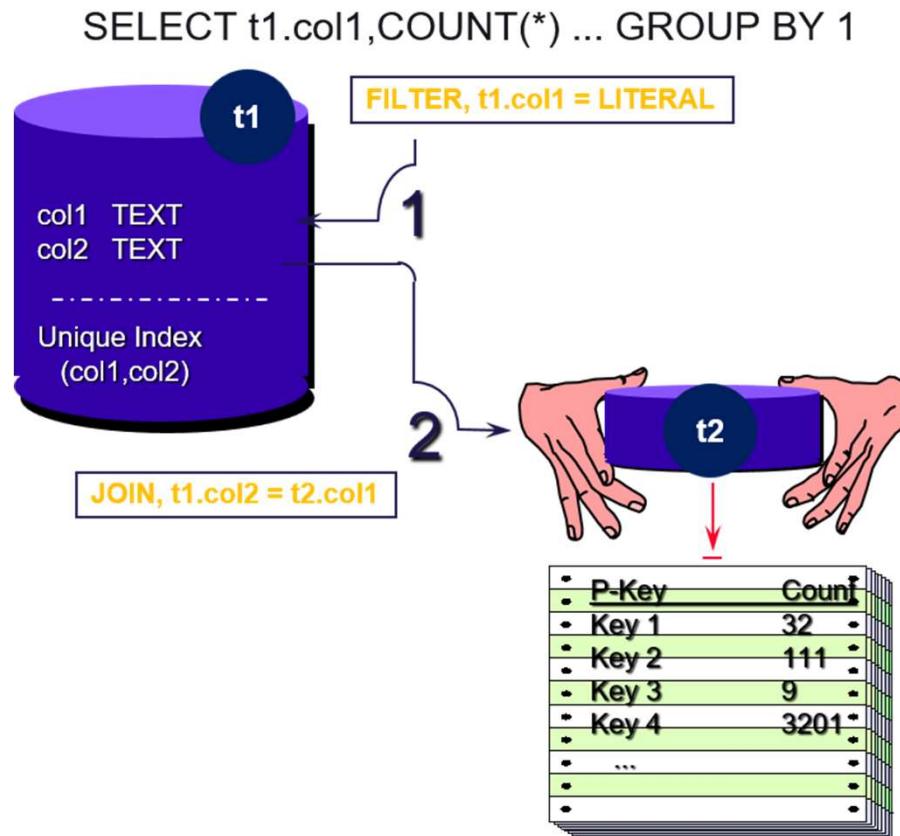
any SQL SELECT statement that was accessing just the leading column(s) of this index for filters or join conditions would actually run faster if the index was not used; if the index was dropped or altered

about unique constraints



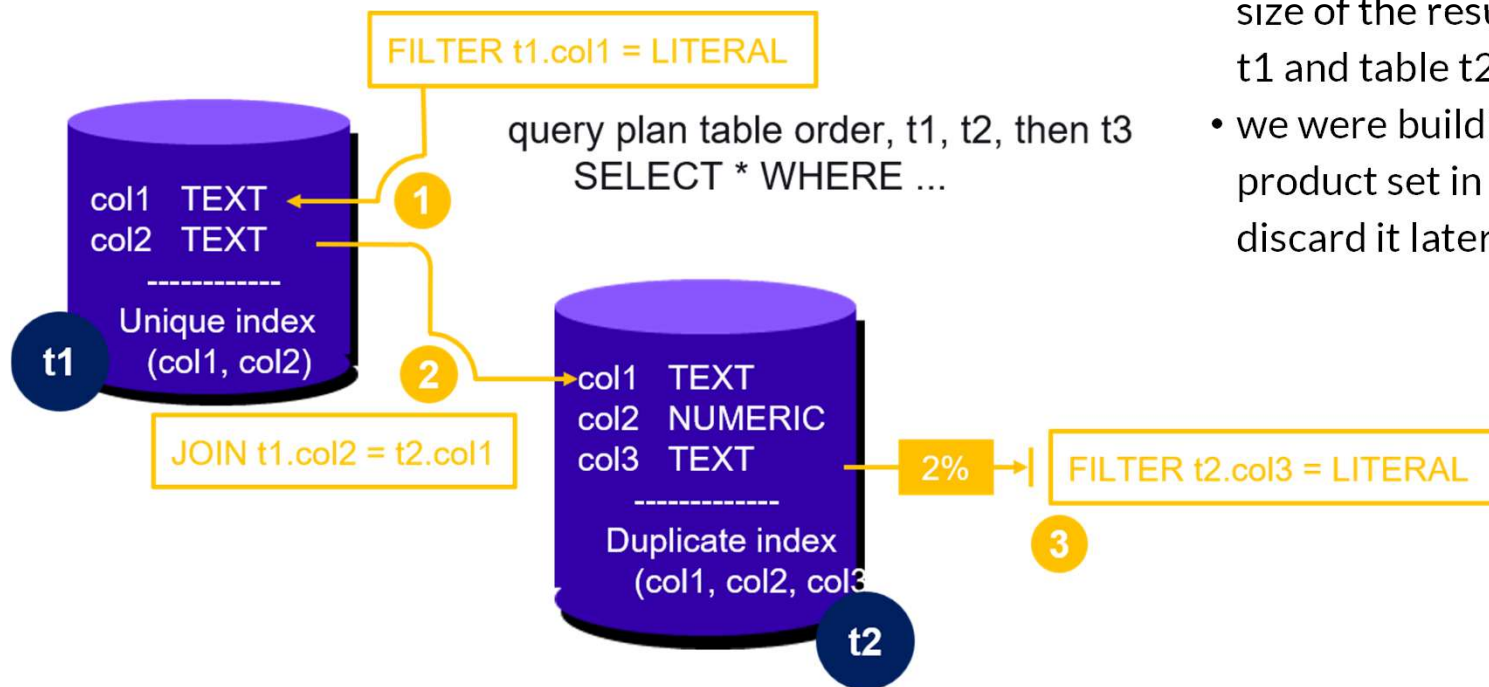
- the column order of a unique constraint can be modified without violating the constraint

example 07: measure join relations, cardinality



- compress out table t2 via an SQL GROUP BY clause. get counts of the join pairs
- this query returned the primary key from table t1 and a count of the join pairs from table t2
- this query informed us that anywhere between eight and 4000 rows were joined from table t2 to each single row from table t1, producing an intermediate product set of way too many rows

example 07: more filter measurement



- the filter condition on table t2 reduced the size of the resultant data set between table t1 and table t2 to 700 rows
- we were building a large intermediate product set in the midst of the join, only to discard it later

example 07

solution

example 07: solution

- the query plan just detailed was the ~*best* available (rbo → cbo)
 - reading table t2 first is best, but that causes a default sequential scan against table t1
 - query run times will never (allow) a secondary table to be sequential (will not allow repetitive/nested sequential scans)
 - query run times will build a run time only index (hash) against t1 just for the processing of this one query. faster, but not zero seconds
- the original query plan is so bad, you have to use query optimizer hints in order to replicate
 - cbo will build the needed index for us

example 07: running the really bad query

```
SET enable_hashjoin = off;
SET enable_mergejoin = off;
SET enable_seqscan = off;

/*+
   Leading(((t1 t2) t3))
   NestLoop(t1 t2)
   NestLoop(t1 t2 t3)
   IndexScan(t1 t1_pkey)
   IndexScan(t2 idx_t2_col1_col2_col3)
   IndexScan(t3 t3_pkey)
*/

EXPLAIN (ANALYZE, BUFFERS)
SELECT t1.col1, t1.col2, t2.col3, t3.col1
FROM t1, t2, t3
WHERE t1.col1 = 'LA'
      AND t1.col2 = t2.col1
      AND t2.col3 = 'FLT-00025'
      AND t2.col3 = t3.col1;
```


example 07: query plan for the really bad query

QUERY PLAN

```
-----
Nested Loop  (cost=90.36..50107.88 rows=1890 width=38)
    (actual time=11.741..382.782 rows=1900 loops=1)
    ->  YB Batched Nested Loop Join  (cost=69.27..50063.05 rows=1890 width=28)
        (actual time=10.834..380.368 rows=1900 loops=1)
        Join Filter: (t1.col2 = t2.col1)
        ->  Index Scan using t1_pkey on t1  (cost=21.09..6873.07 rows=68591 width=18)
            (actual time=2.297..19.531 rows=68600 loops=1)
            Index Cond: (col1 = 'LA'::text)
        ->  Index Scan using idx_t2_col1_col2_col3 on t2  (cost=48.18..644.40 rows=31 width=25)
            (actual time=4.886..4.894 rows=28 loops=67)
            Index Cond: ((col1 = ANY (ARRAY[t1.col2, $1, $2, ..., $1023])) AND
                (col3 = 'FLT-00025'::text))
    ->  Materialize  (cost=21.09..21.21 rows=1 width=10) (actual time=0.001..0.001 rows=1 loops=1900)
        ->  Index Scan using t3_pkey on t3  (cost=21.09..21.20 rows=1 width=10)
            (actual time=0.893..0.895 rows=1 loops=1)
            Index Cond: (col1 = 'FLT-00025'::text)

Planning Time: 31.813 ms
Execution Time: 383.740 ms
Peak Memory Usage: 2049 kB
```

example 07: query plan for the really bad query

- table t1 first – index Scan using t1_pkey on t1 has loops=1
 - t1 is scanned exactly once, filtered by col1 = 'LA', producing all 68,600 matching rows
 - this is the outermost driving table because it is the only index supported filter
- table t2 second – Index Scan using idx_t2_col1_col2_col3 on t2 has loops=67
 - t2 is probed once per batch of t1.col2 values (the YB Batched Nested Loop Join packs up to ~1024 values per batch; 68,600 rows / ~1024 ≈ 67 batches)
 - this is the Join Filter: (t1.col2 = t2.col1) step – t2 is joined against t1's output
- table t3 last – Materialize -> Index Scan using t3_pkey on t3 has loops=1900
 - probed once per surviving row that made it through the t1⋈t2 join (1,900 of them)
 - materialize caches the single result row after the first real lookup, so the other 1,899 loops are free – but logically
- t3 is joined last, against the combined t1⋈t2 result
- the plan drives from t1's weak filter (col1='LA', 98% of the table) first, joins into t2 second, and reaches t3

example 07b: remove the hints, allow full (modern) cbo

QUERY PLAN

```
-----
Nested Loop  (cost=63.27..6811.52 rows=1890 width=38) (actual time=26.163..41.345 rows=1900 loops=1)
->  Index Scan using t3_pkey on t3  (cost=21.09..21.20 rows=1 width=10)
      (actual time=0.314..0.316 rows=1 loops=1)
      Index Cond: (col1 = 'FLT-00025'::text)
->  YB Batched Nested Loop Join  (cost=42.18..6771.41 rows=1890 width=28)
      (actual time=25.845..40.651 rows=1900 loops=1)
      Join Filter: (t1.col2 = t2.col1)
      ->  Seq Scan on t2  (cost=21.09..5917.47 rows=1929 width=25)
            (actual time=20.370..30.504 rows=1900 loops=1)
            Storage Filter: (col3 = 'FLT-00025'::text)
      ->  Index Scan using t1_pkey on t1  (cost=21.09..440.77 rows=1003 width=18)
            (actual time=4.141..4.295 rows=816 loops=2)
            Index Cond: ((col2 = ANY (ARRAY[t2.col1, $1, $2, ..., $1023])) AND (col1 = 'LA'::text))
Planning Time: 113.674 ms
Execution Time: 42.254 ms
Peak Memory Usage: 1575 kB
```

-- original/bad plan
Planning Time: 31.813 ms
Execution Time: 383.740 ms
Peak Memory Usage: 2049 kB

example 07b: remove the hints, allow full (modern) cbo

- table t3 first – Index Scan using t3_pkey on t3 has loops=1
 - scanned once, filtered by the literal col1 = 'FLT-00025', returning its single matching row
 - this is the outermost/driving step – and notice it needs no join filter at all, because the optimizer recognized the transitive equality (t2.col3 = 'FLT-00025' AND t2.col3 = t3.col1 implies t3.col1 = 'FLT-00025' too) and just resolved t3 independently via the constant
- table t2 second – Seq Scan on t2 has loops=1
 - scanned once, with col3 = 'FLT-00025' applied as a Storage Filter, producing the 1,900 candidate rows
 - this sits as the outer side of the YB Batched Nested Loop Join
- t1 last – Index Scan using t1_pkey on t1 has loops=2
 - probed in batches of t2.col1 values (1,900 rows batched into ~1024-sized groups ≈ 2 loops), with Index Cond: (col2 = ANY (ARRAY[...]) AND col1 = 'LA')
 - both the join key and the 'LA' filter combined into one seek
- this is the natural, unhinted plan (07b) – it drives from t3's cheap point lookup first, then t2, then t1 last – the reverse of the FROM-clause order (t1, t2, t3) and the reverse of the hinted 07 plan
- this reversal, chosen entirely by the cost-based optimizer with no hints involved, is exactly why this query runs in ~42ms instead of ~380ms

example 07c: but an even faster plan (flip the indexes over)

```
CREATE TABLE t4 (                                -- replacement for table t1, so table t1 still exists
  col1  TEXT,
  col2  TEXT,
  col3  INT,
  col4  TEXT,
  col5  TEXT,
  col6  TIMESTAMP,
  col7  NUMERIC(10,2),
  col8  TEXT,
  PRIMARY KEY (col2 HASH, col1 ASC)
);

CREATE TABLE t5 (                                -- replacement for table t2, so table t2 still exists
  id    BIGINT,                                   -- table t3 needed no change
  col1  TEXT,
  col2  SMALLINT,
  col3  TEXT,
  PRIMARY KEY (id HASH)
);
CREATE INDEX idx_t5_col3_col2_col1 ON t5 (col3 HASH, col2 ASC, col1 ASC);
```

example 07c: uses the original/unchanged query

QUERY PLAN

```
-----
Nested Loop  (cost=66.27..739.72 rows=1914 width=38) (actual time=7.587..12.984 rows=1900 loops=1)
->  Index Scan using t3_pkey on t3  (cost=21.09..21.20 rows=1 width=10)
      (actual time=0.268..0.269 rows=1 loops=1)
      Index Cond: (col1 = 'FLT-00025'::text)
->  YB Batched Nested Loop Join  (cost=45.18..699.38 rows=1914 width=28)
      (actual time=7.315..12.277 rows=1900 loops=1)
      Join Filter: (t4.col2 = t5.col1)
      ->  Index Only Scan using idx_t5_col3_col2_col1 on t5  (cost=24.09..207.29 rows=1951 width=25)
            (actual time=0.447..0.889 rows=1900 loops=1)
            Index Cond: (col3 = 'FLT-00025'::text)
            Heap Fetches: 0
      ->  Index Scan using t4_pkey on t4  (cost=21.09..245.72 rows=1005 width=18)
            (actual time=4.865..5.000 rows=700 loops=2)
            Index Cond: ((col2 = ANY (ARRAY[t5.col1, $1, $2, ..., $1023])) AND (col1 = 'LA'::text))
```

Planning Time: 113.633 ms

Execution Time: 16.132 ms

Peak Memory Usage: 1641 kB

-- original/bad plan

Planning Time: 31.813 ms

Execution Time: 383.740 ms

Peak Memory Usage: 2049 kB

-- example 07b

Planning Time: 113.674 ms

Execution Time: 42.254 ms

Peak Memory Usage: 1575 kB

example 07c: cbo, and reversed indexes

- table t3 first – index Scan using t3_pkey on t3 has loops=1
 - resolved once via the literal col1 = 'FLT-00025'
 - same transitive-equality reasoning as before (no join filter needed against t2/t1 at this step)
 - table t2 second – index Only Scan using idx_t2_col3_col2_col1 on t5 has loops=1
 - scanned once, and this time col3 = 'FLT-00025' is a genuine Index Cond (not a Storage Filter like t2 was) – plus Heap Fetches: 0, meaning it's fully satisfied from the index alone
 - this is the reordered index doing its job.
 - t1 last – Index Scan using t1_pkey on t4 has loops=2: probed in batches of t2.col1 values, with Index Cond: (col2 = ANY (ARRAY[...]) AND col1 = 'LA') combining the join key and the 'LA' filter into one seek
-
- table order t3 → t2 → t1 – the same driving order as the natural 07b plan (t3 → t2 → t1), just with the reordered-index tables substituted in
 - the only structural difference from 07b is t2's access method: Index Only Scan with a true Index Cond instead of Seq Scan with a Storage Filter
 - the ~3x speedup (07b's ~42ms → this plan's ~16ms) coming entirely from the index column reorder, not from any change in join order

example 07

hands on exercise

- exercise 07
- exercise 07b
- exercise 07c

example 07 (and 01 – 06)

lessons learned

- in the real world, this query took 1+ hours to solve, and there could be 100(s) of these queries
- creation of a "query harness"
 - what is this
 - why is it needed
 - how is it used throughout the entire lifecycle of an application

is schema synonymous with data model ?

schema: [see Data Model](#)

relational model-
attribute
[description of](#)
diagramming
entity
join
normalizing data
projection
rules for defining tables,
rows, and columns
table-relationships

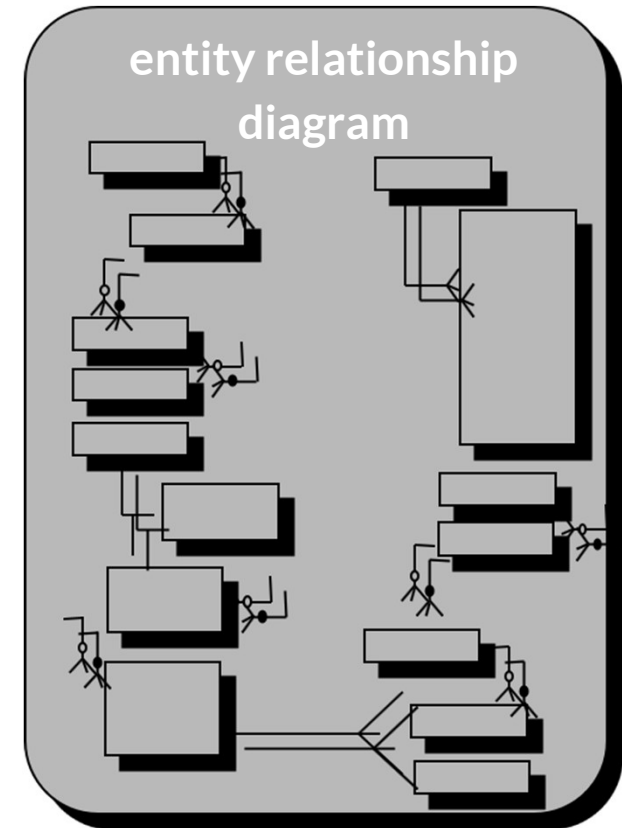
data model-
attribute
defining relationships
description of
[see also Relational Model](#)
table-relationships

[Description of Relational Model:](#)

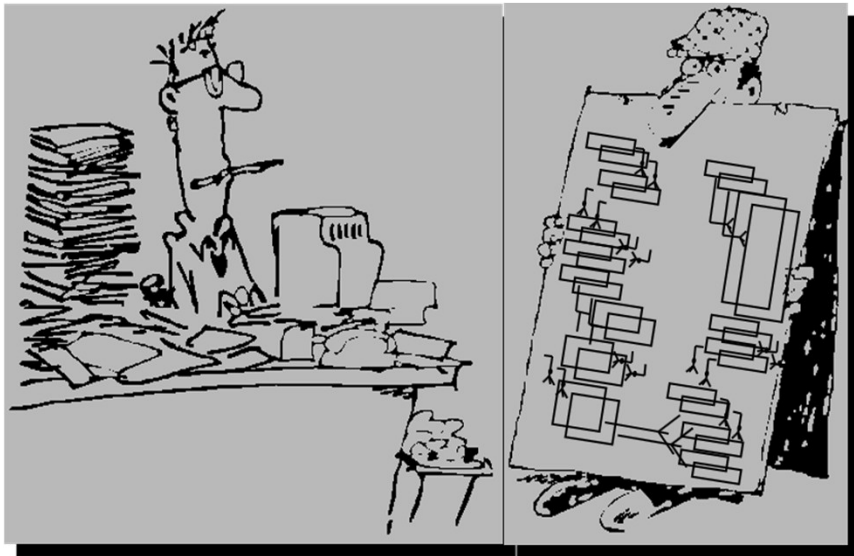
" .. based on the relational calculus devised by E.F. Codd. In practical terms, it means that all data is presented in the form of tables comprising rows and columns. "

steps to create a data model

- identify principle data objects, (entities)
- define relationships between entities
- identify the attributes found in each entity
- create an entity relationship diagram
- translate the entity relationship diagram into relational constructs, (tables and columns)
- resolve complex entity relationships; One to one's, many to many's, etc
- check for compliance with Third Normal Form

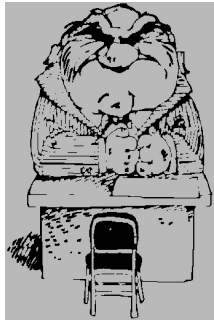


does the data model tell you



	what are the steps to add a new customer order ?
	what is the SQL SELECT statement for the report that I have to write ?
	what is the anticipated performance of my mission critical SQL services ?
	what is the plan to shelf data from this system and how do I access it later ?

common database project time line



end user discovery

requirements
analysis
system
specification



data model

architectural
design
detailed system
specification



coding and debugging



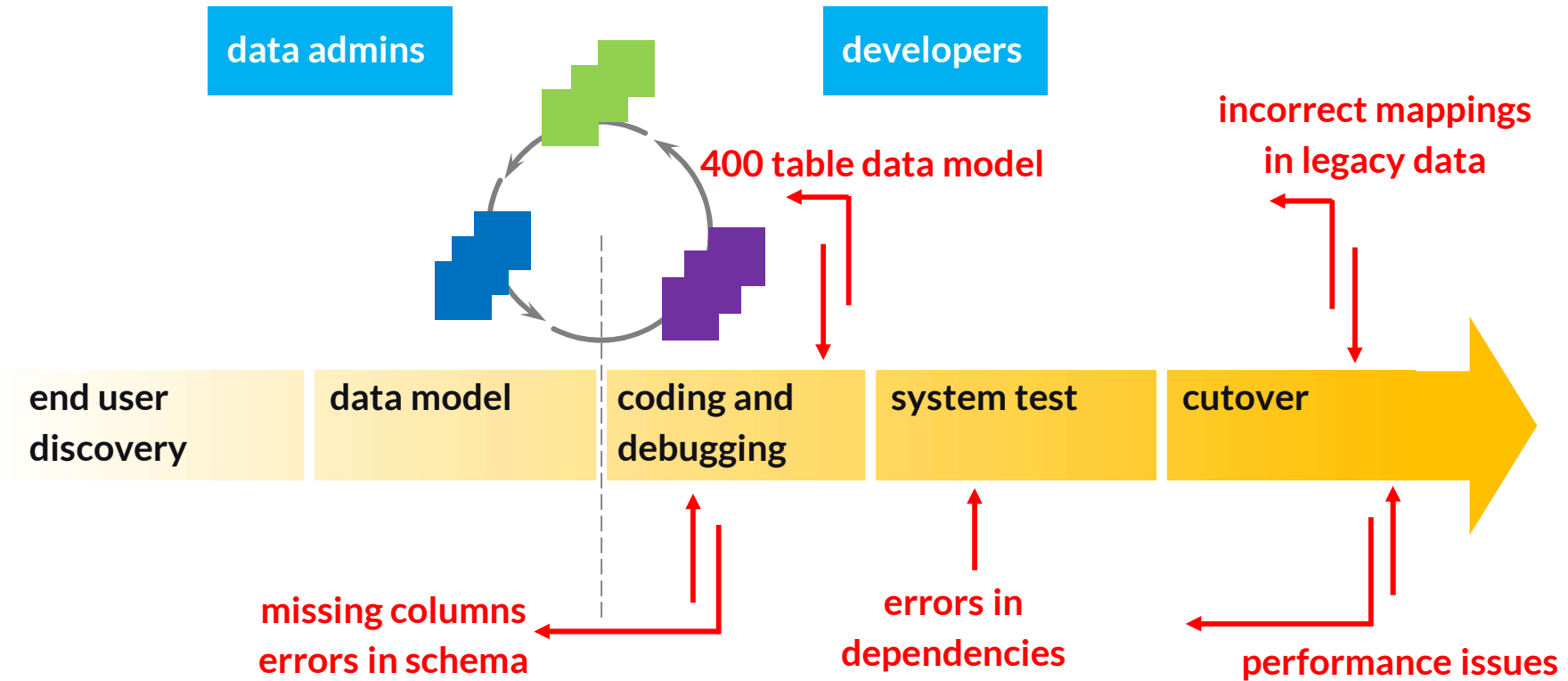
system test



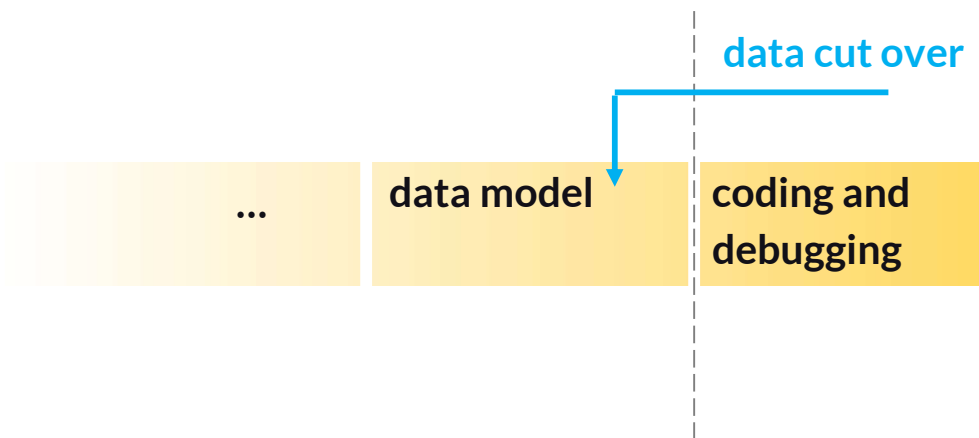
cutover

migration
integration

common sources of error

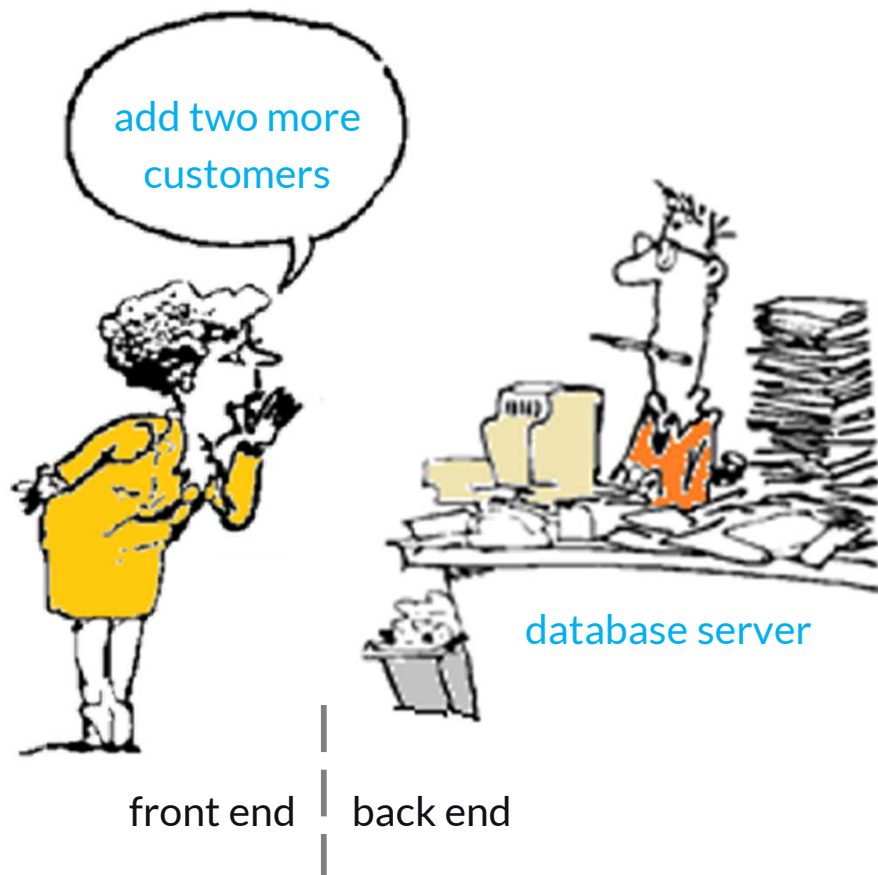


modified time line: early cut over



- legacy system data is loaded into the new system as soon as the data model is completed
- loading the legacy data will uncover forgotten conditions and usages of legacy fields
- development is performed with "production size" data
- conversion scripts may be completed by engineers who are not on the "critical path"
- allows for early measurement of the amount of work and difficulties of migrating

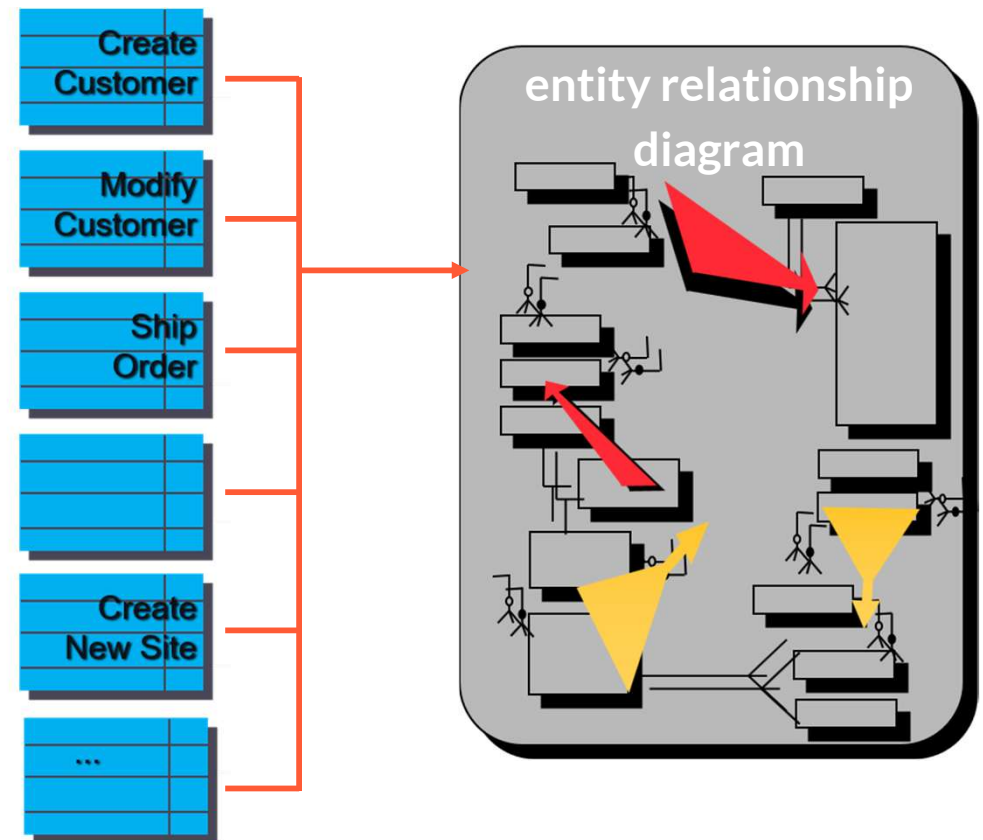
modified time line: process model



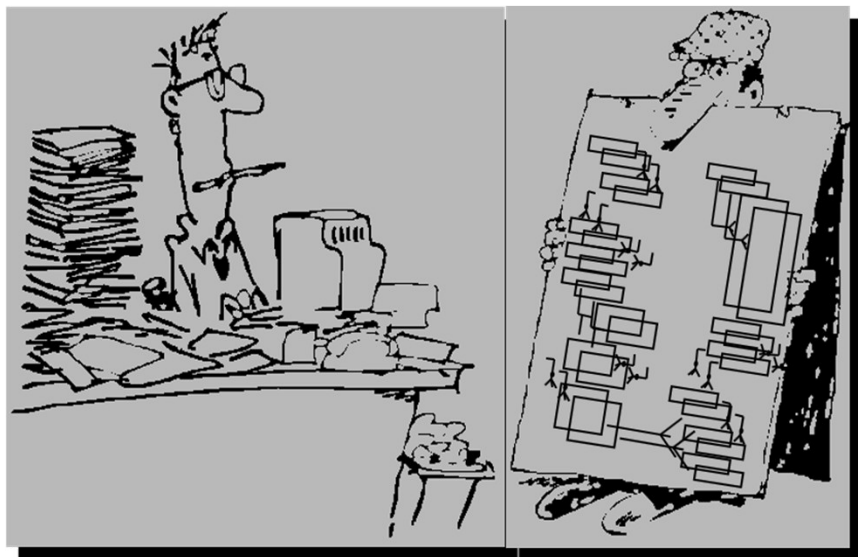
- a complete data model is created from the careful review of the business being performed
- a process model is a record of the information that the data administrators should already have attained
- the final product of a process model is a set of SQL API calls
- the application developer is given the set of SQL API calls. the developer does not have to interpret a 400 table schema
- no more embedded SQL statements

steps to create a process model

- start with the data model
- list each program deliverable to be found in the application system
- list every program event witnessed by each program deliverable
- author the SQL DML statements associated with each program event
- create SQL API calls for each distinct business function



does the data process model tell you



what are the steps to add a new customer order ?



what is the SQL SELECT statement for the report that I have to write ?

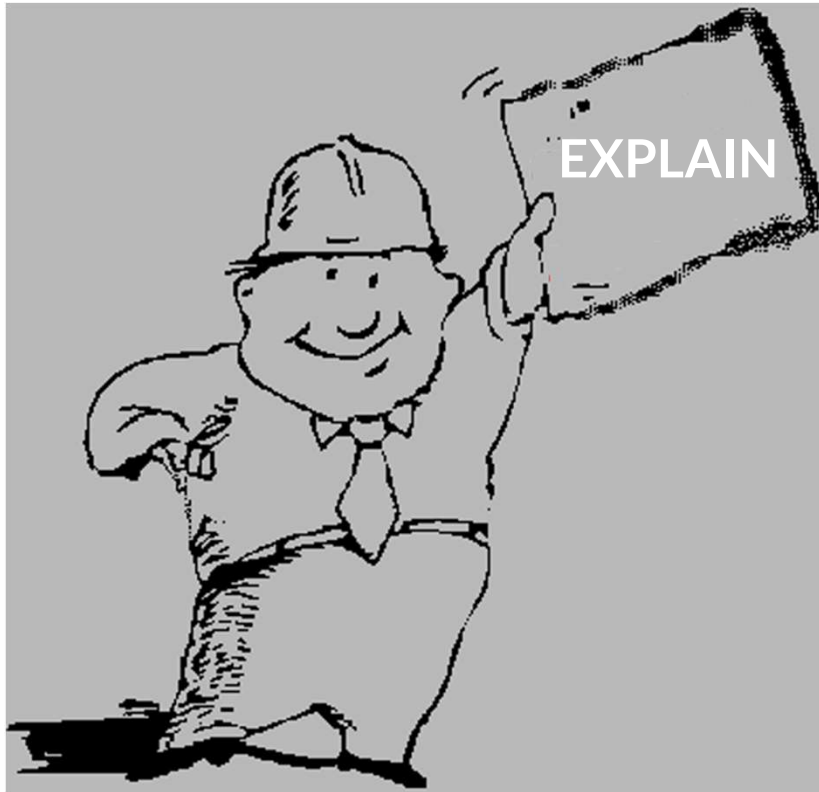


what is the anticipated performance of my mission critical SQL services ?



what is the plan to shelf data from this system and how do I access it later ?

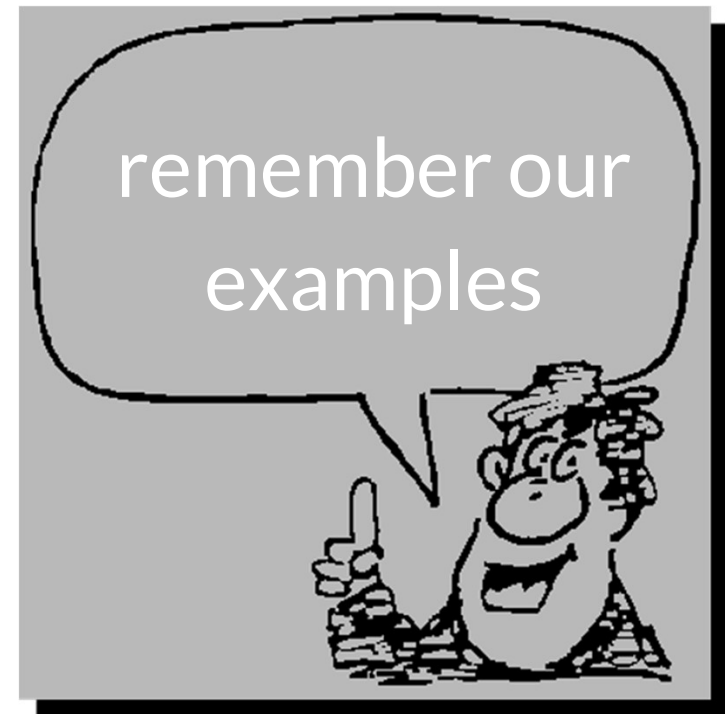
modified time line: performance testing



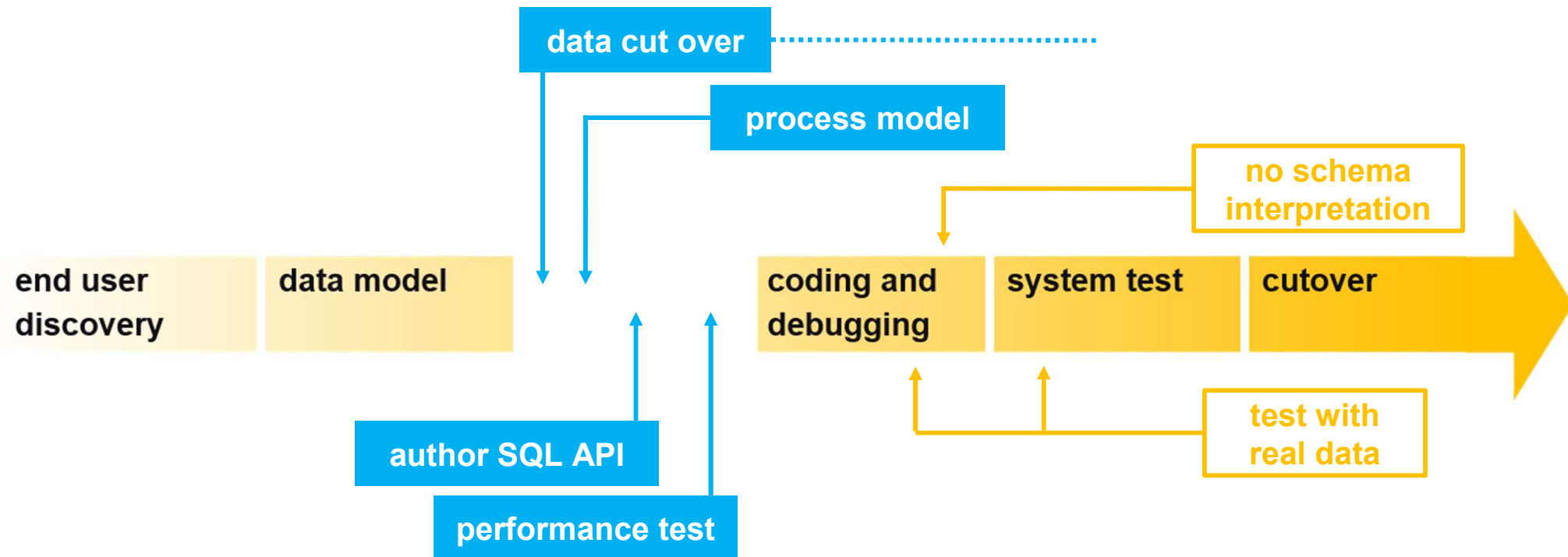
- SQL offers a unique capability; One SQL SELECT statement can effectively model the performance load of an entire report program
- execute the authored SQL DML statements with the anticipated frequencies, concurrency and spread
- check mission critical SQL services for acceptable performance
- use SQL optimization techniques to increase performance before any application deliverables are created

modified time line: SQL optimization

- besides the obvious host and database server tuning, there is one area of SQL programming that very few engineers ever become expert at; the query optimizer
- understanding the query optimizer can be crucial. one 60 byte SQL SELECT statement could initiate 5M i/o calls. minor structural modifications to that SQL SELECT statement could greatly impact performance



modified project time line: final



sources of error, revisited

source of error	cost	correction
400 table schema	programmer productivity, errors in programs	programmer is handed a set of API calls
errors in data model, missing columns	changes to data model cause redevelopment	successful creation of API validates data model
performance	delays, redevelopment/refactoring	model is performance tested prior to development
incorrect map of legacy data	creeping functionality	legacy data is checked early in process
errors in dependencies	higher maintenance, testing	API calls provide SQL programming layer

query plan management (qpm)

automating query monitoring, alerting on query plan regressions

- history of, before qpm-
 - log_min_duration_statement
 - pg_stat_activity
 - pg_stat_statements
 - application performance monitoring, APM
 - active session history, ASH
- query plan management (qpm)
 - why-
 - details; how

history of, before qpm-

- log_min_duration_statement
 - 2003, PostgreSQL 7.4
 - file based, statement by statement
 - only completed statements
 - no aggregation across nodes
- pg_stat_activity
 - early/core PostgreSQL, changed much over the years
 - system table, memory based only
 - what is running right now, ephemeral
 - no aggregation across nodes
- ...
- pg_stat_statements
 - extension, 2009, PostgreSQL 8.4
 - system table, memory based only
 - cumulative since last reset, does persist across clean restarts
 - no aggregation across nodes
- every execution of every distinct normalized query shape — not sampled — bounded by a capacity limit
- pg_stat_statements.max (default 5000 distinct query shapes)

pg_stat_statements, example execution

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

```
SELECT
    LEFT(query, 100) AS statement,
    calls,
    mean_exec_time   AS avg_ms,
    max_exec_time    AS max_ms,
    total_exec_time  AS total_ms,
    rows
FROM pg_stat_statements
WHERE query ILIKE 'select%'
ORDER BY mean_exec_time DESC
LIMIT 10;
```

pg_stat_statements, example output

statement	calls	avg_ms	max_ms	total_ms	rows
SELECT t1.col1, t1.col2, t2.col3, t3.col1 FROM t1, t2, t3 WHERE t1.col1 = \$1 AND t1.col2 = t2.col1	2	108.120848	175.159477	216.2416960	3800
SELECT t4.col1, t4.col2, t5.col3, t3.col1 FROM t4, t5, t3 WHERE t4.col1 = \$1 AND t4.col2 = t5.col1	1	14.766957	14.766957	14.7669570	1900
SELECT LEFT(query, \$1) AS statement, calls, mean_exec_time AS avg_ms, max_exec_time	1	0.477873	0.477873	0.4778730	7
SELECT pg_stat_statements_reset()	1	0.098783	0.098783	0.0987830	1
SELECT LEFT(query, \$1) AS statement, calls, mean_exec_time AS avg_ms, max_exec_time	0	0	0	0	0

application performance monitoring (APM)

functionally different, an application API, at 3 layers-

- the SDK/tracing API your application code calls (`tracer.start_as_current_span(...)`) — what creates the actual trace data
- OTLP (OpenTelemetry Protocol) — the wire format/API used to ship completed spans out of your process to a collector or backend (I used `ConsoleSpanExporter` above just to print locally; swap it for `OTLPSpanExporter` pointed at Datadog/Honeycomb/Jaeger and nothing else in the code changes)
- the backend's own query API — once ingested, you search/visualize traces via that vendor's own REST/GraphQL API (or their UI, which calls the same API underneath)

APM, sample output

Offset (ms)	Event	
0.00	http.request	- request.received
8.17	http.request	- request.routed_to_handler
8.26	app.handler	- span starts
8.29	db.connection.acquire	- pool.checkout_start
14.72	db.connection.acquire	- pool.checkout_complete [6.43ms spent]
14.97	db.query.execute	- query.sent
251.41	db.query.execute	- query.first_response [236.44ms REAL SQL TIME]
252.87	db.query.execute	- query.rows_fetched [1.46ms fetch]
255.81	app.serialize_response	- serialize.start
358.74	app.serialize_response	- serialize.complete [102.93ms serialize]
359.03	app.handler	- span ends
363.17	http.request	- response.sent
363.19	http.request	- span ends (total round trip)

active session history (ASH)

- enabled via `yb_enable_ash` (confirmed on by default on this cluster), it periodically samples every active session cluster-wide and records what it was doing
- data lands in a real, queryable system view: `yb_active_session_history` — columns include `sample_time`, `wait_event_component`, `wait_event_class`, `wait_event`, `wait_event_type`, `query_id`, `pid`, and more
- sampling cadence and retention are tunable: `yb_ash_sampling_interval_ms` (1000ms here — one sample per second), `yb_ash_sample_size` (500 samples per component per sampling event), `yb_ash_circular_buffer_size` (64MB here) — a fixed-size ring buffer, so old samples age out rather than growing unbounded
- bridges the gap `pg_stat_activity` and `pg_stat_statements` leave open: `pg_stat_activity` is live-only (no history once the query ends), `pg_stat_statements` is aggregated (no timeline) — ASH is the one that answers "what was this session doing, sampled over time, even after it's finished".

ASH, sample output

```
SELECT pg_backend_pid() AS my_pid;

SELECT t1.col1, t1.col2,
       (SELECT COUNT(*) FROM t2 WHERE t2.col2 = t1.col1) AS row_count,
       (SELECT SUM(t2.col3) FROM t2 WHERE t2.col2 = t1.col1) AS total_val
FROM t1
WHERE t1.col2 = 'West';
-- my_pid = 1178910
```

ASH, sample output

```
SELECT
  to_char(sample_time, 'HH24:MI:SS') AS time,
  wait_event_component                AS component,
  wait_event_class                    AS class,
  wait_event                          AS event,
  wait_event_type                     AS type
FROM yb_active_session_history
WHERE pid = 1178910
ORDER BY sample_time;
```

ASH, sample output

Output:

time	component	class	event	type
11:10:41	YSQL	TServerWait	IndexRead	RPCWait
11:10:42	YSQL	TServerWait	IndexRead	RPCWait
11:10:43	YSQL	TServerWait	IndexRead	RPCWait
11:10:44	YSQL	TServerWait	TableRead	RPCWait
11:10:45	YSQL	TServerWait	TableRead	RPCWait
11:10:46	YSQL	TServerWait	IndexRead	RPCWait
11:10:47	YSQL	TServerWait	IndexRead	RPCWait
11:10:48	YSQL	TServerWait	IndexRead	RPCWait
11:10:49	YSQL	TServerWait	IndexRead	RPCWait
11:10:50	YSQL	TServerWait	IndexRead	RPCWait
11:10:51	YSQL	YSQLQuery	OnCpu_Active	Cpu
11:10:52	YSQL	TServerWait	IndexRead	RPCWait
11:10:53	YSQL	TServerWait	IndexRead	RPCWait
11:10:54	YSQL	TServerWait	IndexRead	RPCWait
11:10:56	YSQL	TServerWait	IndexRead	RPCWait
11:10:57	YSQL	TServerWait	IndexRead	RPCWait

(16 rows)

ASH, what can you do with this data-

the single most useful thing: it separates “why is this slow” into buckets that call for completely different remedies

- dominated by IndexRead/TableRead (RPCWait) — like our example — means the time is going to network round trips to storage, not computation. the fix is architectural: fewer RPCs (batching, rewriting correlated subqueries as joins — literally what exercises 06→06b→06c did), or covering indexes to avoid extra hops
- dominated by OnCpu_Active — means the time is genuine computation (sorting, aggregating, evaluating expressions). the fix is different: better join/aggregate algorithm choice, work_mem, or parallelism — not batching
- dominated by lock-related wait events — a concurrency problem, not a query-shape problem. points us at transaction design, not indexing
- dominated by client/idle wait — means the database isn’t the bottleneck at all. this is the tell that sends us back to the application/APM side instead of tuning SQL further — the one thing pure database-side tools can’t tell you on their own

query plan management (qpm): why-

- we have thousands and thousands of queries (too many) to design, tune, monitor
 - query planner "cost" is relative across different queries
 - query planner "time"
 - the "month end summary report" query (run once per month, by a daemon) will always be 30 seconds
 - the "log in" query (run 100s of times per second by a live operator) must be very sub-second
 - data volumes grow naturally over time, change isn't automatically an alert
- if the time to complete goes up slowly over months, and the query plan is the same; normal data growth ? we good ?
- if the query plan changes, and time is faster; we good ?
- if the query plan changes, and time is worse by a margin; roll a truck

qpm, configuring

no CREATE EXTENSION needed — it's a core, built-in mechanism (unlike pg_stat_statements), controlled by GUCs)

- `yb_pg_stat_plans_track` — scope of tracking (default all)
- `yb_pg_stat_plans_max_cache_size` — capacity, in distinct query/plan pairs (default 5000)
- `yb_pg_stat_plans_cache_replacement_algorithm` — eviction policy once full (default `simple_clock_lru`)
- `yb_pg_stat_plans_plan_format` — json by default
- `yb_pg_stat_plans_verbose_plans` / `yb_pg_stat_plans_show_max_exec_params` — off by default, extra detail when turned on
- `yb_qpm_compress_text` — compresses stored plan/hint text (on by default)

qpm, accessing

two system views, both queryable directly, no wrapper needed-

- `yb_pg_stat_plans` — one row per distinct (query, plan) pair
 - `queryid`, `planid`, `calls`, `avg_exec_time`, `max_exec_time`, `avg_est_cost`
 - critically a `hints` column containing a complete, replayable `pg_hint_plan`-compatible string describing exactly that plan (join order, access method per table, and the planner GUC settings in effect)
- `yb_pg_stat_plans_insights` — a higher-level view
 - `plan_require_evaluation`, `min_avg_exec_time`, `min_avg_est_cost`
 - specifically flag queries whose current plan is worse than a previously known one — the actual "regression detection" surface

qpm, in use

The single most useful query against this view: find any queryid that has more than one distinct planid — meaning the same logical query has been executed under different plans

```
SELECT queryid, planid, calls, avg_exec_time, avg_est_cost
FROM yb_pg_stat_plans
WHERE queryid IN (
    SELECT queryid FROM yb_pg_stat_plans
    GROUP BY queryid HAVING COUNT(DISTINCT planid) > 1
)
ORDER BY queryid, avg_exec_time DESC;
```

example 07, 07b specifically

the same queryid that merged into one blended row in pg_stat_statements a few turns ago is split into two distinct plans here

```
SELECT
    RIGHT(planid::text, 8)          AS plan_id,
    calls,
    round(avg_exec_time::numeric,1) AS avg_ms,
    round(avg_est_cost::numeric,0)  AS est_cost,
    CASE WHEN hints LIKE '%SeqScan(t2)%' THEN 'SeqScan(t2)'
         WHEN hints LIKE '%IndexOnlyScan(t2%' THEN 'IdxOnlyScan(t2)'
         ELSE 'other' END           AS t2_access
FROM yb_pg_stat_plans
WHERE queryid = 3958159052188978184
ORDER BY avg_exec_time DESC;
```

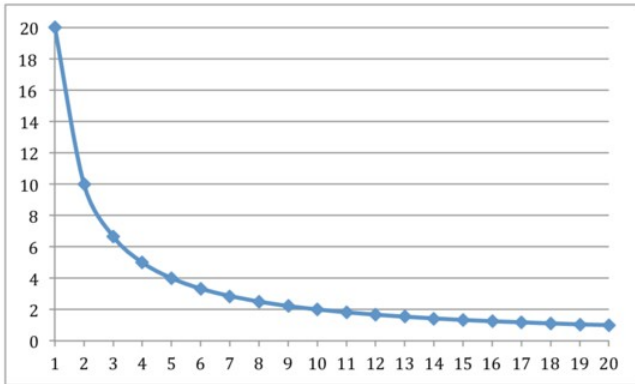
data output from that last query

plan_id	calls	avg_ms	est_cost	t2_access
21637953	1	175.2	10989	IdxOnlyScan(t2)
95128495	1	41.1	6812	SeqScan(t2)

query plan management demonstration:

web program demonstration, pre-chat

zipfian data distributions, query plans, the driver



zipfian data distribution

data: 500K total rows

500 "typical" customers with ~100 rows each

3 "mega" customers with 150,000 rows each (90% of the table)

query, prepared statement, real application using connection pool/driver-

```
PREPARE stmt(int) AS SELECT *  
FROM qo_demo_zipf WHERE customer_id = $1;
```

repeatedly EXECUTE stmt(XXX customer_id), a typical customer
then a mega customer

zipfian data distributions, query plans, the driver

why/how it breaks-

- Postgres/YSQL doesn't replan a prepared statement from scratch on every call
- first few executions it uses a "custom" plan tailored to the actual bound value each time
- correctly picking an Index Scan for a typical customer (cheap, ~100 matching rows)
- and a Seq Scan for a mega customer (cheap relative to indexing through 150,000 individual entries)

starting around the 3rd execution, the planner evaluates switching to one reusable "generic" plan that doesn't know which value is coming next

Index Scan is the safe bet, and locks that plan in for every future call on that connection

references:

generic versus customer plans, <https://www.postgresql.org/docs/current/sql-prepare.html>

controlling this behavior, <https://www.postgresql.org/docs/current/runtime-config-query.html>

more reading, https://yugabytedb.tips/taming-prepared-statement-re-planning-with-plan_cache_mode-in-yugabytedb/

query plan management demonstration:

YUGABYTEDB FEATURE DEMONSTRATION

Query plan management

Overview

Query

Query plan management

Alerts

Query

Ready

Watch a prepared statement's cached plan silently stop fitting the data it's asked about.

In our data: A table t1 with 500,000 rows split into two very different populations under one customer_id column — 500 "typical" customers with ~100 rows each (50,000 rows total), and 3 "mega" customers with 150,000 rows each (450,000 rows total, ~90% of the table). Indexed on customer_id.

The query: a single prepared statement, held open on one persistent connection the way a pooled application would — `PREPARE find_by_customer(int) AS SELECT id, customer_id, payload FROM t1 WHERE customer_id = $1;` — then repeatedly `EXECUTE find_by_customer(<id>)`. Click **Typical** a few times, then click **Mega**: Postgres's planner will have already settled on one reusable plan shaped by the typical calls, and the mega customer's query pays for it.

Query for:

☐ Typical customer

☒ Mega customer

Run query

Typical ANY:

0.00 ms

Typical CUSTOM:

0.00 ms

Typical GENERIC:

0.00 ms

Mega ANY:

0.00 ms

Mega CUSTOM:

0.00 ms

Mega GENERIC:

0.00 ms

Results

No query run yet

ID	CUSTOMER_ID	PAYLOAD
Run a query to see the first five rows returned.		

Effective plan_cache_mode

auto

Set plan_cache_mode:

☒ auto

☐ force_custom_plan

☐ force_generic_plan

Most recent query plan

Run a query to see its plan here.

checkpoint

- why was the example 07 query initially slow
- why should you do a volume data test after creating the data model, and before entering the application development phase
- what is a zipfian data distribution
- generic|custom query plans through the driver; what do we need to know here

Next step ..

- —
- —
- —
- —



end

query optimizer; what constitutes a query (execution) plan

- predicate pushdown, filter placement
 - deciding where in the plan tree to apply a WHERE-clause filter, ideally as early as possible to reduce **tuple counts** before expensive operations
- aggregation strategy
 - hash aggregate vs. sort-then-group-aggregate for GROUP BY
- sort elimination
 - recognizing when an index already delivers tuples in the order an ORDER BY or GROUP BY needs, avoiding an explicit sort step entirely
- subquery transformation
 - rewriting a correlated subquery into a join (or vice versa), converting IN/EXISTS into semi-joins/anti-joins, or "unnesting" subqueries into the main query when it's provably equivalent and cheaper
- materialization versus pipelining
 - rewriting a correlated subquery into a join (or vice versa), converting IN/EXISTS into semi-joins/anti-joins, or "unnesting" subqueries into the main query when it's provably equivalent and cheaper

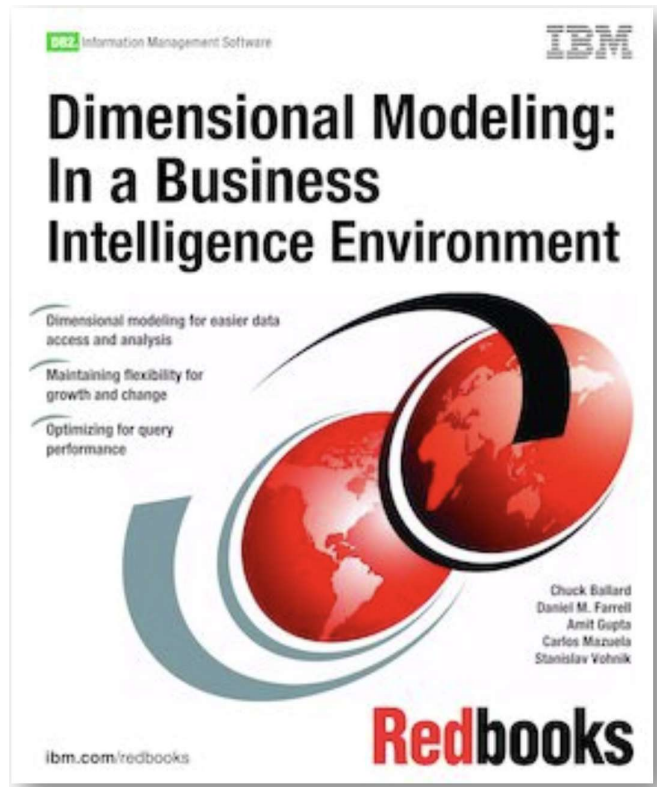
...

query optimizer; what constitutes a query (execution) plan

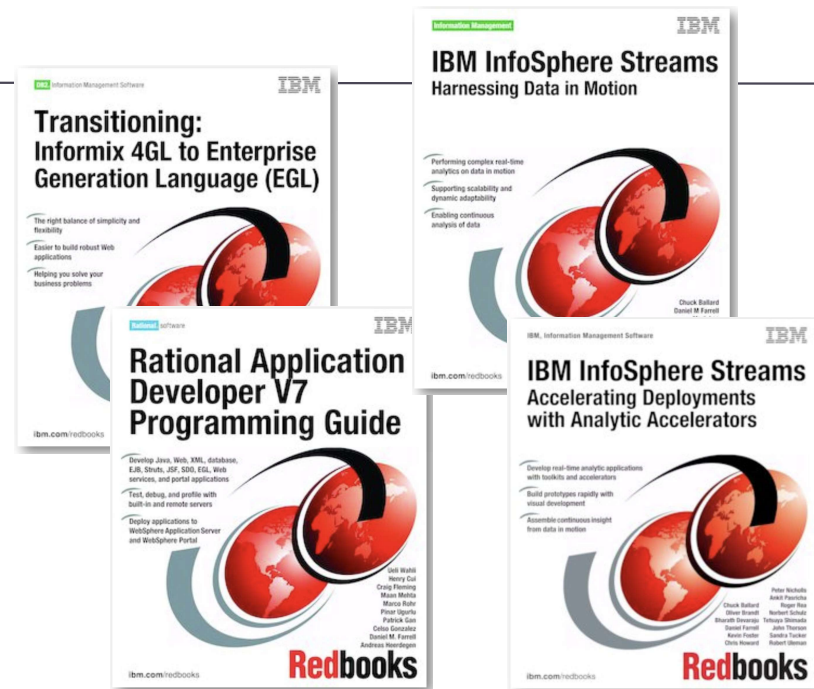
...

- parallel execution planning
 - how many worker processes to use for a parallel scan/join/aggregate, and whether parallelism is even worth the coordination overhead for this particular query
- partition pruning
 - or partitioned tables, determining which partitions can be skipped entirely based on the query's filter conditions
- constants folding, expression simplification
 - or partitioned tables, determining which partitions can be skipped entirely based on the query's filter conditions
- view/CTE inlining
 - whether to expand a view or CTE inline into the surrounding query (allowing cross-boundary optimization) or treat it as an optimization fence (materializing it separately)

if you need further text-



Dimensional Modeling, <https://www.redbooks.ibm.com/abstracts/sg247138.html>



in detail: yb_pg_stat_plans

Column	Type	Collation	Nullable	Default
dbid	oid			
userid	oid			
queryid	bigint			
planid	bigint			
first_used	timestamp with time zone			
last_used	timestamp with time zone			
hints	text			
calls	bigint			
avg_exec_time	double precision			
max_exec_time	double precision			
max_exec_time_params	text			
avg_est_cost	double precision			
plan	text			

in detail: yb_pg_stat_plans_insights

Column	Type	Collation	Nullable	Default
dbid	oid			
userid	oid			
queryid	bigint			
planid	bigint			
first_used	timestamp with time zone			
last_used	timestamp with time zone			
hints	text			
avg_exec_time	double precision			
avg_est_cost	double precision			
min_avg_exec_time	double precision			
min_avg_est_cost	double precision			
plan_require_evaluation	text			
plan_min_exec_time	text			

how are queryid and planid calculated-

- queryid: hash code to identify identical normalized queries
 - normalized meaning literal constants are stripped out
 - sensitive to syntactic structure, not just semantic structure
 - FROM t1, t0 gets a different queryid than FROM t0, t1, and $a1=a0$ differs from $a0=a1$, even though those pairs are logically identical queries so same shape really means same parse tree, not same meaning
- planid: hash of a query's execution plan representation
 - hash of the plan itself, confirming your read but it's order-insensitive in exactly the ways queryid isn't
 - the docs specifically say Plan ID doesn't care about the order of FROM-list items, AND-clause order, or operand order in an equality ($a1=a2$ vs $a2=a1$)
 - hashing the plan's actual shape (which access methods, which join strategy, which operators) rather than the surface syntax that produced it

another look at the alert query-

```
WITH ordered AS (  
    SELECT  
        queryid,  
        planid,  
        avg_exec_time,  
        last_used,  
        LAG(planid) OVER (PARTITION BY queryid ORDER BY last_used) AS prev_planid,  
        LAG(avg_exec_time) OVER (PARTITION BY queryid ORDER BY last_used) AS prev_avg_exec_time,  
        ROW_NUMBER() OVER (PARTITION BY queryid ORDER BY last_used DESC) AS rn  
    FROM yb_pg_stat_plans  
)  
SELECT queryid, prev_planid, planid AS current_planid,  
    round(prev_avg_exec_time::numeric, 2) AS prev_avg_ms,  
    round(avg_exec_time::numeric, 2) AS current_avg_ms,  
    round(((avg_exec_time - prev_avg_exec_time) / prev_avg_exec_time * 100)::numeric, 1) AS pct_slower  
FROM ordered  
WHERE rn = 1  
    AND prev_planid IS NOT NULL  
    AND planid <> prev_planid  
    AND avg_exec_time > prev_avg_exec_time * 1.2  
ORDER BY pct_slower DESC;
```

-- the currently active plan, per query
-- there was a prior plan to compare to
-- the plan actually changed
-- and it's ≥20% slower

another look at the alert query-

queryid	prev_planid	current_planid	prev_avg_ms	current_avg_ms	pct_slower
6498551768429791526	-5152680836455757205	3645420288158387274	3.23	490.83	15111.7